

Chapter 23. The Z File System (ZFS)

Table of Contents

- 23.1. What Makes ZFS Different
- 23.2. Quick Start Guide
- 23.3. `zpool` Administration
- 23.4. `zfs` Administration
- 23.5. Delegated Administration
- 23.6. ZFS Native Encryption
- 23.7. Boot Environments
- 23.8. Advanced Topics
- 23.9. Further Resources
- 23.10. ZFS Features and Terminology

ZFS is an advanced file system designed to solve major problems found in previous storage subsystem software.

Originally developed at Sun Microsystems™, ongoing open source ZFS development has moved to the [OpenZFS Project](#).

ZFS has three major design goals:

- Data integrity: All data includes a [checksum](#) of the data. ZFS calculates checksums and writes them along with the data. When reading that data later, ZFS recalculates the checksums. If the checksums do not match, meaning detecting one or more data errors, ZFS will attempt to automatically correct errors when ditto-, mirror-, or parity-blocks are available.
- Pooled storage: adding physical storage devices to a pool, and allocating storage space from that shared pool. Space is available to all file systems and volumes, and increases by adding new storage devices to the pool.
- Performance: caching mechanisms provide increased performance. [ARC](#) is an advanced memory-based read cache. ZFS provides a second level disk-based read cache with [L2ARC](#), and a disk-based synchronous write cache named [ZIL](#).

A complete list of features and terminology is in [ZFS Features and Terminology](#).



23.1. What Makes ZFS Different

More than a file system, ZFS is fundamentally different from traditional file systems. Combining the traditionally separate roles of volume manager and file system provides ZFS with unique advantages. The file system is now aware of the underlying structure of the disks. Traditional file systems could exist on a single disk at a time. If there were two disks then creating two separate file systems was necessary. A traditional hardware RAID configuration avoided this problem by presenting the operating system with a single logical disk made up of the space provided by physical disks on top of which the operating system placed a file system. Even with software RAID solutions like those provided by GEOM, the UFS file system living on top of the RAID believes it is dealing with a single device. ZFS' combination of the volume manager and the file system solves this and allows the creation of file systems that all share a pool of available storage. One big advantage of ZFS' awareness of the physical disk layout is that existing file systems grow automatically when adding extra disks to the pool. This new space then becomes available to the file systems. ZFS can also apply different properties to each file system. This makes it useful to create separate file systems and datasets instead of a single monolithic file system.

23.2. Quick Start Guide

The FreeBSD installer can install the system directly onto a ZFS pool, a configuration known as Root-on-ZFS; see [Guided Root-on-ZFS](#). This section shows how to create and manage additional ZFS pools and datasets on a running system.

FreeBSD can mount ZFS pools and datasets during system initialization. To enable it, run:

```
# sysrc zfs_enable="YES"
```



This adds `zfs_enable="YES"` to `/etc/rc.conf`. Running `service zfs enable` makes the same change.

Then start the service:

```
# service zfs start
```



The examples in this section assume three SATA disks with the device names **ada0**, **ada1**, and **ada2**. Users of SCSI/SAS hardware should instead use **da** device names, and users of NVMe storage **nda** device names.

23.2.1. Single Disk Pool

Create a GPT partition on an empty disk first. Then add a partition of type `freebsd-zfs` from which a single, non-redundant pool is created:





```
# gpart create -s gpt ada1
# gpart add -t freebsd-zfs ada1
# zpool create example /dev/ada1p1
```

To view the new pool, review the output of `df`:



```
# df /example
```

Filesystem	1K-blocks	Used	Avail	Capacity	Mounted on
example	17547136	0	17547136	0%	/example

This output shows creating and mounting of the `example` pool, and that it is now accessible as a file system. Create files for users to browse:



```
# cd /example
# ls
# touch testfile
# ls -al
```

	total	4							
drwxr-xr-x	2	root	wheel	3	Aug	29	23:15	.	
drwxr-xr-x	21	root	wheel	512	Aug	29	23:12	..	
-rw-r--r--	1	root	wheel	0	Aug	29	23:15	testfile	

This pool is not using any advanced ZFS features and properties yet. To create a dataset on this pool with compression enabled:



```
# zfs create example/compressed
# zfs set compression=gzip example/compressed
```

The `example/compressed` dataset is now a ZFS compressed file system. Try copying some large files to `/example/compressed`.

Disable compression with:



```
# zfs set compression=off example/compressed
```

To unmount a file system, use `zfs umount` and then verify with `df`:



```
# zfs umount example/compressed
# df
```



Filesystem	1K-blocks	Used	Avail	Capacity	Mounted on
example	17547008	0	17547008	0%	/example

To re-mount the file system to make it accessible again, use `zfs mount` and verify with `df`:

```
# zfs mount example/compressed
# df
```

Filesystem	1K-blocks	Used	Avail	Capacity	Mounted on
example	17547008	0	17547008	0%	/example
example/compressed	17547008	0	17547008	0%	/example/compressed

Running `mount` shows the pool and file systems:

```
# mount
/dev/ada0p1 on / (ufs, local)
devfs on /dev (devfs, local)
/dev/ada0p2 on /usr (ufs, local, soft-updates)
example on /example (zfs, local)
example/compressed on /example/compressed (zfs, local)
```

Use ZFS datasets like any file system after creation. Set other available features on a per-dataset basis when needed. The example below creates a new file system called `data`. It assumes the file system contains important files and configures it to store two copies of each data block.

```
# zfs create example/data
# zfs set copies=2 example/data
```

Use `df` to see the data and space usage:

```
# df
```

Filesystem	1K-blocks	Used	Avail	Capacity	Mounted on
/dev/ada0p1	2026030	235234	1628714	13%	/
devfs	1	1	0	100%	/dev
/dev/ada0p2	54098308	1032864	48737580	2%	/usr
example	17547008	0	17547008	0%	/example
example/compressed	17547008	0	17547008	0%	/example/compressed
example/data	17547008	0	17547008	0%	/example/data

Notice that all file systems in the pool have the same available space. Using `df` in these examples shows that the file systems use the space they need and all draw from the same pool. ZFS gets rid of concepts such as volumes and partitions, and allows several file systems to share the same pool. 

To destroy the file systems and then the pool that is no longer needed:

```
# zfs destroy example/compressed
# zfs destroy example/data
# zpool destroy example
```

23.2.2. RAID-Z

Disks fail. One way to avoid data loss from disk failure is to use RAID. ZFS supports this feature in its pool design. RAID-Z pools require three or more disks but provide more usable space than mirrored pools.

This example creates a RAID-Z pool. First the GPT partitions are created on each disk, then specify these GPT partitions to add to the pool using the `raidz` keyword:

```
# gpart create -s gpt ada0
# gpart add -t freebsd-zfs ada0
# gpart create -s gpt ada1
# gpart add -t freebsd-zfs ada1
# gpart create -s gpt ada2
# gpart add -t freebsd-zfs ada2
# zpool create storage raidz ada0p1 ada1p1 ada2p1
```

i Note

Keeping the number of devices used in a RAID-Z configuration between three and nine is recommended. For environments requiring a single pool consisting of 10 disks or more, consider breaking it up into smaller RAID-Z groups. If two disks are available, ZFS mirroring provides redundancy if required. Refer to [zpool\(8\)](#) for more details.

The previous example created the `storage` zpool. This example makes a new file system called `home` in that pool:

```
# zfs create storage/home
```

Enable compression and store an extra copy of directories and files:

```
# zfs set copies=2 storage/home
# zfs set compression=gzip storage/home
```

To make this the new home directory for users, copy the user data to this directory and create the appropriate symbolic links:

```
# cp -rp /home/* /storage/home
# rm -rf /home /usr/home
# ln -s /storage/home /home
# ln -s /storage/home /usr/home
```

User data is now stored on the freshly-created **/storage/home**. Test by adding a new user and logging in as that user.

Create a file system snapshot to roll back to later:

```
# zfs snapshot storage/home@2026-07-11
```

ZFS creates snapshots of a dataset, not a single directory or file.

The `@` character is a delimiter between the file system or volume name and the snapshot name. Before deleting an important directory, back up the file system, then roll back to an earlier snapshot in which the directory still exists:

```
# zfs rollback storage/home@2026-07-11
```

To list all available snapshots, run `ls` in the file system's **.zfs/snapshot** directory. For example, to see the snapshot taken:

```
# ls /storage/home/.zfs/snapshot
```

Write a script to take regular snapshots of user data. Over time, snapshots can use up a lot of disk space. Remove the previous snapshot using the command:

```
# zfs destroy storage/home@2026-07-11
```

After testing, make **/storage/home** the real **/home** with this command:

```
# zfs set mountpoint=/home storage/home
```

Run `df` and `mount` to confirm that the system now treats the file system as the real **/home**:





```
# mount
/dev/ada0p1 on / (ufs, local)
devfs on /dev (devfs, local)
/dev/ada0p2 on /usr (ufs, local, soft-updates)
storage on /storage (zfs, local)
storage/home on /home (zfs, local)

# df
Filesystem      1K-blocks    Used   Avail Capacity  Mounted on
/dev/ada0p1      2026030  235240  1628708    13%      /
devfs              1         1         0   100%    /dev
/dev/ada0p2      54098308 1032826  48737618     2%    /usr
storage          26320512     0  26320512     0%    /storage
storage/home     26320512     0  26320512     0%    /home
```

This completes the RAID-Z configuration. Add daily status updates about the created file systems to the nightly [periodic\(8\)](#) runs by adding this line to **/etc/periodic.conf**:

```
daily_status_zfs_enable="YES"
```

[periodic\(8\)](#) can also run scheduled scrubs of the pool; see [Scrubbing a Pool](#).

23.2.3. Recovering RAID-Z

Every software RAID has a method of monitoring its **state**. View the status of RAID-Z devices using:



```
# zpool status -x
```

If all pools are [Online](#) and everything is normal, the message shows:



```
all pools are healthy
```

If there is a problem, perhaps a disk being in the [Offline](#) state, the pool state will look like this:



```
pool: storage
state: DEGRADED
status: One or more devices has been taken offline by the administrator.
        Sufficient replicas exist for the pool to continue functioning in a
        degraded state.
action: Online the device using 'zpool online' or replace the device with
        'zpool replace'.
scan: none requested
```



config:

NAME	STATE	READ	WRITE	CKSUM
storage	DEGRADED	0	0	0
raidz1-0	DEGRADED	0	0	0
ada0p1	ONLINE	0	0	0
ada1p1	OFFLINE	0	0	0
ada2p1	ONLINE	0	0	0

errors: No known data errors

"OFFLINE" shows the administrator took **ada1p1** offline using:

```
# zpool offline storage ada1p1
```

Power down the computer now and replace **ada1p1**. Recreate the GPT partition of type `freebsd-zfs` on the new disk. Power up the computer and return **ada1p1** to the pool:

```
# zpool replace storage ada1p1
```

Next, check the status again, this time without `-x` to display all pools:

```
# zpool status storage
pool: storage
state: ONLINE
scan: resilvered 3.21G in 00:04:36 with 0 errors on Sat Jul 11 10:32:19 2026
config:
```

NAME	STATE	READ	WRITE	CKSUM
storage	ONLINE	0	0	0
raidz1-0	ONLINE	0	0	0
ada0p1	ONLINE	0	0	0
ada1p1	ONLINE	0	0	0
ada2p1	ONLINE	0	0	0

errors: No known data errors

In this example, everything is normal.

23.2.4. Data Verification

ZFS uses checksums to verify the integrity of stored data. Creating file systems automatically enables them.



⚠ Warning

Disabling Checksums is possible but *not* recommended! Checksums take little storage space and provide data integrity. Most ZFS features will not work properly with checksums disabled. Disabling these checksums will not increase performance noticeably.

Verifying the data checksums (called *scrubbing*) ensures integrity of the `storage` pool with:

```
# zpool scrub storage
```

The duration of a scrub depends on the amount of data stored. Larger amounts of data will take proportionally longer to verify. Since scrubbing is I/O intensive, ZFS allows a single scrub to run on each pool at a time. After scrubbing completes, view the status with `zpool status`:

```
# zpool status storage
pool: storage
state: ONLINE
  scan: scrub repaired 0B in 00:19:16 with 0 errors on Sat Jul 11 10:32:19 2026
config:

   NAME        STATE        READ  WRITE CKSUM
   storage     ONLINE         0     0     0
     raidz1-0   ONLINE         0     0     0
       ada0p1   ONLINE         0     0     0
       ada1p1   ONLINE         0     0     0
       ada2p1   ONLINE         0     0     0

errors: No known data errors
```

Displaying the completion date of the last scrubbing helps decide when to start another. Routine scrubs help protect data from silent corruption and ensure the integrity of the pool.

Refer to [zfs\(8\)](#) and [zpool\(8\)](#) for other ZFS options.

23.3. `zpool` Administration

ZFS administration uses two main utilities. The `zpool` utility controls the operation of the pool and allows adding, removing, replacing, and managing disks. The `zfs` utility allows creating, destroying, and managing datasets, both [file systems](#) and [volumes](#).

23.3.1. Creating and Destroying Storage Pools



The most important decision when creating a ZFS storage pool is which types of vdevs to group the physical disks into. See the list of [vdev types](#) for details about the possible options. The vdev types determine the redundancy, capacity, and performance characteristics of a pool. A pool's layout is not fixed forever at creation time. Mirrors allow adding new disks to the vdev, and stripes upgrade to mirrors by attaching a new disk to the vdev. RAID-Z vdevs grow one disk at a time with [RAID-Z expansion](#), which requires OpenZFS 2.3 or later, first available in FreeBSD 15.0. Adding new vdevs expands a pool at any time, and [removing top-level vdevs](#) is possible within limits. Reshaping a live pool takes time and I/O bandwidth, so choosing suitable vdev types up front remains important.

Create a simple mirror pool on disks containing GPT partitions of type `freebsd-zfs`:

```
# zpool create mypool mirror /dev/ada1p1 /dev/ada2p1
# zpool status
  pool: mypool
  state: ONLINE
 config:

      NAME            STATE        READ  WRITE CKSUM
  mypool             ONLINE         0     0     0
    mirror-0         ONLINE         0     0     0
      ada1p1         ONLINE         0     0     0
      ada2p1         ONLINE         0     0     0

errors: No known data errors
```

To create more than one vdev with a single command, specify groups of disks separated by the vdev type keyword, `mirror` in this example:

```
# zpool create mypool mirror /dev/ada1p1 /dev/ada2p1 mirror /dev/ada3p1 /dev/ada4p1
# zpool status
  pool: mypool
  state: ONLINE
 config:

      NAME            STATE        READ  WRITE CKSUM
  mypool             ONLINE         0     0     0
    mirror-0         ONLINE         0     0     0
      ada1p1         ONLINE         0     0     0
      ada2p1         ONLINE         0     0     0
    mirror-1         ONLINE         0     0     0
      ada3p1         ONLINE         0     0     0
      ada4p1         ONLINE         0     0     0

errors: No known data errors
```

Pools can also use partitions rather than whole disks. Putting ZFS in a separate partition allows the same disk to have other partitions for other purposes. In particular, it allows adding partitions with bootcode and file systems needed for booting. This allows booting from disks that are also members of a pool. ZFS adds no performance penalty on FreeBSD when using a partition rather than a whole disk. Using partitions also allows the administrator to *under-provision* the disks, using less than the full capacity. If a future replacement disk of the same nominal size as the original actually has a slightly smaller capacity, the smaller partition will still fit, using the replacement disk.

Create a [RAID-Z2](#) pool using partitions:

```
# zpool create mypool raidz2 /dev/ada0p1 /dev/ada1p1 /dev/ada2p1 /dev/ada3p1 /dev/ada4p1 /dev/ada5p1
# zpool status
  pool: mypool
  state: ONLINE
 config:

    NAME                STATE        READ  WRITE CKSUM
    mypool               ONLINE             0     0     0
      raidz2-0          ONLINE             0     0     0
        ada0p1          ONLINE             0     0     0
        ada1p1          ONLINE             0     0     0
        ada2p1          ONLINE             0     0     0
        ada3p1          ONLINE             0     0     0
        ada4p1          ONLINE             0     0     0
        ada5p1          ONLINE             0     0     0

errors: No known data errors
```

ZFS aligns and sizes its smallest writes to each vdev based on the vdev's `ashift` value, the base 2 logarithm of the sector size. OpenZFS detects the sector size the disks report when creating a vdev and chooses `ashift` accordingly. Some drives report 512-byte sectors for compatibility while using 4096-byte sectors internally. Create pools on such drives with an explicit `-o ashift=12` to force 4096-byte alignment:

```
# zpool create -o ashift=12 mypool mirror /dev/ada1p1 /dev/ada2p1
```

Setting the `sysctl(8)` variable `vfs.zfs.vdev.min_auto_ashift` to `12` gives the same result for pool creation and for later `zpool add` and `zpool attach` operations. `ashift` is a property of each vdev, not of the pool as a whole. It is fixed when creating the vdev and cannot change afterwards, so verify it before committing data to a pool.

Destroy a pool that is no longer needed to reuse the disks. Destroying a pool requires unmounting the file systems in that pool first. If any dataset is in use, the unmount operation fails without



destroying the pool. Force the pool destruction with `-f`. This can cause undefined behavior in applications which had open files on those datasets.

23.3.2. Pool Properties

Like datasets, pools have properties that report status and control behavior. Display every property of a pool with `zpool get all mypool`, or name specific properties:

```
# zpool get health,capacity mypool
NAME      PROPERTY  VALUE    SOURCE
mypool    health    ONLINE   -
mypool    capacity  27%      -
```

Change a writable property with `zpool set`:

```
# zpool set comment="Backup pool" mypool
# zpool get comment mypool
NAME      PROPERTY  VALUE      SOURCE
mypool    comment   Backup pool local
```

Properties like `size`, `capacity`, `fragmentation`, and `health` are read-only status values. Others change how the pool behaves; this chapter uses `autoexpand` to grow a pool automatically ([Growing a Pool](#)), `autoreplace` with hot spares ([Hot Spares and Automatic Replacement with zfsd](#)), `autotrim` on SSD pools ([TRIM and Initialization](#)), and `compatibility` for portable pools ([Upgrading a Storage Pool](#)). Set properties at pool creation time by passing `-o` to `zpool create`, as shown with `ashift` above. [zpoolprops\(7\)](#) describes every pool property.

23.3.3. Adding and Removing Devices

Two ways exist for adding disks to a pool: attaching a disk to an existing vdev with `zpool attach`, or adding vdevs to the pool with `zpool add`. Some [vdev types](#) allow adding disks to the vdev after creation. RAID-Z vdevs accept new disks only through [RAID-Z expansion](#).

A pool created with a single disk lacks redundancy. It can detect corruption but can not repair it, because there is no other copy of the data. The `copies` property may be able to recover from a small failure such as a bad sector, but does not provide the same level of protection as mirroring or RAID-Z. Starting with a pool consisting of a single disk vdev, use `zpool attach` to add a new disk to the vdev, creating a mirror. Also use `zpool attach` to add new disks to a mirror group, increasing redundancy and read performance. When partitioning the disks used for the pool, replicate the layout of the first disk on to the second. Use `gpart backup` and `gpart restore` to make this process easier.

Upgrade the single disk (stripe) vdev **ada0p3** to a mirror by attaching **ada1p3**:





```
# zpool status
pool: mypool
state: ONLINE
config:

    NAME      STATE    READ WRITE CKSUM
    mypool    ONLINE      0     0     0
        ada0p3  ONLINE      0     0     0

errors: No known data errors
# zpool attach mypool ada0p3 ada1p3
Make sure to wait until resilver is done before rebooting.
# gpart bootcode -b /boot/pmbr -p /boot/gptzfsboot -i 1 ada1
partcode written to ada1p1
bootcode written to ada1
# zpool status
pool: mypool
state: ONLINE
status: One or more devices is currently being resilvered. The pool will
continue to function, possibly in a degraded state.
action: Wait for the resilver to complete.
scan: resilver in progress since Sat Jul 11 11:15:25 2026
738M scanned at 105M/s, 522M issued at 74.6M/s, 781M total
522M resilvered, 66.84% done, 00:00:03 to go
config:

    NAME      STATE    READ WRITE CKSUM
    mypool    ONLINE      0     0     0
        mirror-0  ONLINE      0     0     0
            ada0p3  ONLINE      0     0     0
            ada1p3  ONLINE      0     0     0 (resilvering)

errors: No known data errors
# zpool status
pool: mypool
state: ONLINE
scan: resilvered 781M in 00:00:16 with 0 errors on Sat Jul 11 11:18:36 2026
config:

    NAME      STATE    READ WRITE CKSUM
    mypool    ONLINE      0     0     0
        mirror-0  ONLINE      0     0     0
            ada0p3  ONLINE      0     0     0
            ada1p3  ONLINE      0     0     0

errors: No known data errors
```



When adding disks to the existing vdev is not an option, an alternative method is to add another vdev to the pool. Adding vdevs provides higher performance by distributing writes across the vdevs. Each vdev provides its own redundancy. Mixing vdev types like `mirror` and `RAID-Z` is possible but discouraged. Adding a non-redundant vdev to a pool containing mirror or RAID-Z vdevs risks the data on the entire pool. Distributing writes means a failure of the non-redundant disk will result in the loss of a fraction of every block written to the pool.

ZFS stripes data across each of the vdevs. For example, with two mirror vdevs, this is effectively a RAID 10 that stripes writes across two sets of mirrors. ZFS allocates space so that each vdev reaches 100% full at the same time. Having vdevs with different amounts of free space will lower performance, as more data writes go to the less full vdev.

When attaching new devices to a boot pool, remember to update the bootcode.

Attach a second mirror group (**ada2p3** and **ada3p3**) to the existing mirror:

```
# zpool status
pool: mypool
state: ONLINE
scan: resilvered 781M in 00:00:16 with 0 errors on Sat Jul 11 12:11:56 2026
config:
```

NAME	STATE	READ	WRITE	CKSUM
mypool	ONLINE	0	0	0
mirror-0	ONLINE	0	0	0
ada0p3	ONLINE	0	0	0
ada1p3	ONLINE	0	0	0

```
errors: No known data errors
```

```
# zpool add mypool mirror ada2p3 ada3p3
```

```
# gpart bootcode -b /boot/pmbr -p /boot/gptzfsboot -i 1 ada2
```

```
partcode written to ada2p1
```

```
bootcode written to ada2
```

```
# gpart bootcode -b /boot/pmbr -p /boot/gptzfsboot -i 1 ada3
```

```
partcode written to ada3p1
```

```
bootcode written to ada3
```

```
# zpool status
```

```
pool: mypool
```

```
state: ONLINE
```

```
scan: resilvered 781M in 00:00:16 with 0 errors on Sat Jul 11 09:58:28 2026
```

```
config:
```

NAME	STATE	READ	WRITE	CKSUM
mypool	ONLINE	0	0	0
mirror-0	ONLINE	0	0	0
ada0p3	ONLINE	0	0	0
ada1p3	ONLINE	0	0	0
mirror-1	ONLINE	0	0	0
ada2p3	ONLINE	0	0	0

```
ada3p3  ONLINE      0      0      0
```

```
errors: No known data errors
```

`zpool add` also attaches dedicated log and cache devices to an existing pool. A [log](#) vdev stores the ZFS intent log on separate low-latency storage, accelerating synchronous writes such as those issued by databases and NFS. Mirror log devices, because losing an unmirrored log device together with a system crash costs the pool the last few seconds of synchronous writes; see [Synchronous Writes, the ZIL, and SLOG](#) for sizing and tuning guidance. A [cache](#) vdev extends the [ARC](#) with a second level of read cache on fast storage. Cache devices need no redundancy, as ZFS reads any block that fails to read from the cache from the original pool disks instead. The contents of the cache, the [L2ARC](#), survive reboots by default.

Add a mirrored log vdev and an NVMe cache device to an existing pool:

```
# zpool add mypool log mirror ada4p2 ada5p2
# zpool add mypool cache nda0p2
# zpool status mypool
  pool: mypool
  state: ONLINE
    scan: resilvered 781M in 00:00:16 with 0 errors on Sat Jul 11 09:58:28 2026
config:
```

NAME	STATE	READ	WRITE	CKSUM
mypool	ONLINE	0	0	0
mirror-0	ONLINE	0	0	0
ada0p3	ONLINE	0	0	0
ada1p3	ONLINE	0	0	0
mirror-1	ONLINE	0	0	0
ada2p3	ONLINE	0	0	0
ada3p3	ONLINE	0	0	0
logs				
mirror-2	ONLINE	0	0	0
ada4p2	ONLINE	0	0	0
ada5p2	ONLINE	0	0	0
cache				
nda0p2	ONLINE	0	0	0

```
errors: No known data errors
```

`zpool detach` removes single disks from a mirror vdev when enough redundancy remains. If a single disk remains in a mirror group, that group ceases to be a mirror and becomes a stripe, risking the entire pool if that remaining disk fails.

Remove a disk from a three-way mirror group:





```
# zpool status
pool: mypool
state: ONLINE
scan: scrub repaired 0B in 00:01:11 with 0 errors on Sat Jul 11 03:14:02 2026
config:
```

NAME	STATE	READ	WRITE	CKSUM
mypool	ONLINE	0	0	0
mirror-0	ONLINE	0	0	0
ada0p3	ONLINE	0	0	0
ada1p3	ONLINE	0	0	0
ada2p3	ONLINE	0	0	0

```
errors: No known data errors
```

```
# zpool detach mypool ada2p3
```

```
# zpool status
pool: mypool
state: ONLINE
scan: scrub repaired 0B in 00:01:11 with 0 errors on Sat Jul 11 03:14:02 2026
config:
```

NAME	STATE	READ	WRITE	CKSUM
mypool	ONLINE	0	0	0
mirror-0	ONLINE	0	0	0
ada0p3	ONLINE	0	0	0
ada1p3	ONLINE	0	0	0

```
errors: No known data errors
```

`zpool remove` removes entire top-level vdevs from a pool. ZFS evacuates the vdev by copying all of its allocated data to the other vdevs in the pool, then detaches the disks. Removal works for hot spares, cache, log, special, and dedup devices, and for data vdevs that are single disks or mirrors. Data vdevs are not removable from pools that contain a top-level RAID-Z vdev, and all top-level vdevs in the pool must use the same `ashift`. The evacuation runs in the background and the pool stays online throughout; monitor progress with `zpool status`. After the removal completes, ZFS keeps an in-memory table mapping the blocks of the removed vdev to their new locations. The table is small but permanent; `zpool remove -n` estimates its memory use before starting a removal.

Remove one of the two mirror vdevs from a pool:



```
# zpool remove mypool mirror-1
# zpool status mypool
pool: mypool
state: ONLINE
scan: scrub repaired 0B in 00:01:11 with 0 errors on Sat Jul 11 03:14:02 2026
```




```

remove: Evacuation of mirror in progress since Sat Jul 11 10:15:31 2026
       1.71G copied out of 2.32G at 111M/s, 73.71% done, 0h0m to go
config:

NAME      STATE    READ WRITE CKSUM
mypool    ONLINE    0     0     0
mirror-0  ONLINE    0     0     0
  ada0p3  ONLINE    0     0     0
  ada1p3  ONLINE    0     0     0
mirror-1  ONLINE    0     0     0
  ada2p3  ONLINE    0     0     0
  ada3p3  ONLINE    0     0     0

errors: No known data errors
# zpool status mypool
  pool: mypool
  state: ONLINE
    scan: scrub repaired 0B in 00:01:11 with 0 errors on Sat Jul 11 03:14:02 2026
remove: Removal of vdev 1 copied 2.32G in 0h0m, completed on Sat Jul 11 10:15:53 2026
       10.9K memory used for removed device mappings
config:

NAME      STATE    READ WRITE CKSUM
mypool    ONLINE    0     0     0
mirror-0  ONLINE    0     0     0
  ada0p3  ONLINE    0     0     0
  ada1p3  ONLINE    0     0     0
indirect-1 ONLINE    0     0     0

errors: No known data errors

```

The `indirect-1` entry is a placeholder for the removed vdev's remapped blocks and contains no disks. Cancel an in-progress removal with `zpool remove -s`.

23.3.4. RAID-Z Expansion

RAID-Z expansion grows an existing RAID-Z vdev by one disk at a time, without changing its parity level.

i Note

RAID-Z expansion requires OpenZFS 2.3 or later, first available in FreeBSD 15.0. Pools created on earlier releases need the `raidz_expansion` feature enabled with `zpool upgrade` before expanding.



To expand a RAID-Z vdev, run `zpool attach` with the name of the RAID-Z vdev as shown by `zpool status` and the new disk:

```
# zpool attach mypool raidz2-0 ada6p3
```

The expansion reflows the existing data across the enlarged set of disks in the background while the pool remains online and in use. Add `-w` to make `zpool attach` wait until the expansion completes. `zpool status` reports progress on the `expand:` line:

```
# zpool status mypool
pool: mypool
state: ONLINE
  scan: scrub repaired 0B in 06:14:36 with 0 errors on Sat Jul 11 04:31:19 2026
expand: expansion of raidz2-0 in progress since Sat Jul 11 09:42:12 2026
       1.83T / 5.36T copied at 186M/s, 34.14% done, 05:31:43 to go
config:

    NAME      STATE    READ WRITE CKSUM
    mypool    ONLINE      0     0     0
      raidz2-0 ONLINE      0     0     0
        ada0p3 ONLINE      0     0     0
        ada1p3 ONLINE      0     0     0
        ada2p3 ONLINE      0     0     0
        ada3p3 ONLINE      0     0     0
        ada4p3 ONLINE      0     0     0
        ada5p3 ONLINE      0     0     0
        ada6p3 ONLINE      0     0     0

errors: No known data errors
```

When the expansion finishes, the capacity of the new disk becomes available:

```
# zpool status mypool | grep expand:
expand: expanded raidz2-0 copied 5.36T in 08:23:35, on Sat Jul 11 18:05:47 2026
```

Blocks written before the expansion keep their old data-to-parity ratio and continue to occupy space accordingly. The full capacity gain from the new disk therefore applies to data written after the expansion, while old data benefits only when rewritten, for example with `zfs rewrite`. Repeat the procedure to grow a vdev by several disks, attaching one at a time.

23.3.5. dRAID Pools



dRAID is a variant of RAID-Z that distributes hot spare capacity across all disks in the vdev. A [dRAID](#) vdev is built from internal RAID-Z groups plus optional *distributed spares*, all spread evenly over every disk. When a disk fails, ZFS rebuilds into the distributed spare by reading from and writing to all remaining disks in parallel, completing in a fraction of the time a resilver onto a single spare disk takes. Replacing the failed disk then rebuilds it from the distributed spare, restoring full protection. dRAID targets pools with many disks, where RAID-Z rebuild times grow dangerously long.

The vdev type spells out the complete layout, per [zpoolconcepts\(7\)](#):

```
draid[parity][:datad][:childrenc][:sparess]
```

parity is the parity level of the internal groups, 1 to 3, defaulting to 1. *data* is the number of data devices per redundancy group, defaulting to 8. *children* is the total number of disks, serving as a cross-check when listing many devices. *spares* is the number of distributed hot spares, defaulting to 0.

Create a pool from a 24-disk dRAID vdev with double parity, 8 data disks per redundancy group, and 2 distributed spares:

```
# zpool create mypool draid2:8d:24c:2s ada0 ada1 ada2 ada3 ada4 ada5 ada6 ada7 ada8 ada9 ada10 ada11
```

The distributed spares appear under `spares` in `zpool status`, with names like `draid2-0-0` for the first spare of the first vdev.

dRAID trades space efficiency for rebuild speed. Unlike RAID-Z, dRAID uses a fixed stripe width, padding smaller writes with zeros. With the default of 8 data disks and 4 KB sectors, the minimum allocation is 32 KB, which reduces usable capacity and compression effectiveness for datasets dominated by small blocks. The capacity of the distributed spares is also committed up front, whether or not a disk ever fails. Pairing a dRAID pool with a [special vdev](#) keeps metadata and small blocks off the wide stripes and recovers much of the lost efficiency. For pools with a handful of disks, RAID-Z remains the better choice.

23.3.6. Special Allocation Classes

Special allocation classes dedicate vdevs to specific types of pool data. A [special](#) vdev stores pool metadata such as indirect blocks and dnodes, and optionally the data blocks of small files. A `dedup` vdev stores the [deduplication](#) tables. Placing this metadata on fast devices such as NVMe mirrors speeds up metadata-heavy operations like `zfs list`, and deduplication lookups on pools of otherwise slower disks.

Add a mirrored special vdev to an existing pool:

```
# zpool add mypool special mirror nda0 nda1
```

New metadata allocations then go to the special vdev; existing metadata stays where it is until rewritten.

Setting the `special_small_blocks` dataset property makes the special vdev store data blocks up to the given size as well:

```
# zfs set special_small_blocks=16K mypool/projects
```

Valid values are zero, which disables storing data blocks in the special class, or a power of two from 512 bytes up to 1 MB on FreeBSD 14.x; FreeBSD 15.0 accepts values up to the maximum block size of 16 MB. Setting the property to the dataset's `recordsize` sends all data blocks of that dataset to the special class. When a special vdev fills up, new allocations spill back to the normal class instead of failing.

Warning

A special vdev is not a cache. It holds the only copy of the pool metadata allocated to it, and losing it destroys the pool. Match the redundancy of special and dedup vdevs to the redundancy of the data vdevs, for example by using a mirror.

Removing a special or dedup vdev with `zpool remove` is possible, subject to the restrictions described in [Adding and Removing Devices](#). In practice, the matching `ashift` requirement often prevents removal when the special vdev uses devices with a different sector size than the data vdevs.

23.3.7. Checking the Status of a Pool

Pool status is important. If a drive goes offline or ZFS detects a read, write, or checksum error, the corresponding error count increases. The `status` output shows the configuration and status of each device in the pool and the status of the entire pool. Actions to take and details about the last `scrub` are also shown.

```
# zpool status
pool: mypool
state: ONLINE
scan: scrub repaired 0B in 02:25:36 with 0 errors on Sat Jul 11 06:51:26 2026
config:
```

NAME	STATE	READ	WRITE	CKSUM
mypool	ONLINE	0	0	0
raidz2-0	ONLINE	0	0	0
ada0p3	ONLINE	0	0	0
ada1p3	ONLINE	0	0	0
ada2p3	ONLINE	0	0	0

ada3p3	ONLINE	0	0	0
ada4p3	ONLINE	0	0	0
ada5p3	ONLINE	0	0	0

errors: No known data errors

Options adjust the output to the task at hand:

- `-v` displays verbose data error information, printing a complete list of the errors found since the last complete pool scrub, including the names of affected files.
- `-x` displays the status of pools with errors or pools that are otherwise unavailable, hiding healthy pools.
- `-s` displays the number of slow I/O operations on each leaf vdev, meaning operations that did not complete within 30 seconds.
- `-e` displays unhealthy vdevs only, keeping the output short for pools with many devices; this flag requires FreeBSD 14.1 or later.
- `-p` displays numbers as exact, parseable values instead of rounded human-readable ones.
- `-t` displays the [TRIM](#) status of each vdev.

Refer to [zpool-status\(8\)](#) for the complete list of options.

23.3.8. Clearing Errors

When detecting an error, ZFS increases the read, write, or checksum error counts. Clear the error message and reset the counts with `zpool clear mypool`. Clearing the error state can be important for automated scripts that alert the administrator when the pool encounters an error. Without clearing old errors, the scripts may fail to report further errors.

23.3.9. Replacing a Functioning Device

It may be desirable to replace one disk with a different disk. When replacing a working disk, the process keeps the old disk online during the replacement. The pool never enters a [degraded](#) state, reducing the risk of data loss. Running `zpool replace` copies the data from the old disk to the new one. After the operation completes, ZFS disconnects the old disk from the vdev. If the new disk is larger than the old disk, it may be possible to grow the zpool, using the new space. See [Growing a Pool](#).

Replace a functioning device in the pool:

```
# zpool status
pool: mypool
state: ONLINE
```



config:

NAME	STATE	READ	WRITE	CKSUM
mypool	ONLINE	0	0	0
mirror-0	ONLINE	0	0	0
ada0p3	ONLINE	0	0	0
ada1p3	ONLINE	0	0	0

errors: No known data errors

zpool replace mypool ada1p3 ada2p3

Make sure to wait **until** resilvering finishes before rebooting.

When booting from the pool 'mypool', update the boot code on the newly attached disk 'ada2p3'.

Assuming GPT partitioning is used and ada0 is the new boot disk, use the following command:

```
gpart bootcode -b /boot/pmbr -p /boot/gptzfsboot -i 1 ada0
# gpart bootcode -b /boot/pmbr -p /boot/gptzfsboot -i 1 ada2
# zpool status
pool: mypool
state: ONLINE
status: One or more devices is currently being resilvered. The pool will
       continue to function, possibly in a degraded state.
action: Wait for the resilver to complete.
scan: resilver in progress since Sat Jul 11 14:21:35 2026
      781M / 781M scanned, 604M / 781M issued at 101M/s
      604M resilvered, 77.39% done, 00:00:01 to go
```

config:

NAME	STATE	READ	WRITE	CKSUM
mypool	ONLINE	0	0	0
mirror-0	ONLINE	0	0	0
ada0p3	ONLINE	0	0	0
replacing-1	ONLINE	0	0	0
ada1p3	ONLINE	0	0	0
ada2p3	ONLINE	0	0	0 (resilvering)

errors: No known data errors

zpool status

```
pool: mypool
state: ONLINE
scan: resilvered 781M in 00:00:17 with 0 errors on Sat Jul 11 14:21:52 2026
```

config:

NAME	STATE	READ	WRITE	CKSUM
mypool	ONLINE	0	0	0
mirror-0	ONLINE	0	0	0
ada0p3	ONLINE	0	0	0
ada2p3	ONLINE	0	0	0



errors: No known data errors

On mirrored vdevs, adding `-s` to `zpool replace` or `zpool attach` performs a *sequential resilver* instead of a healing resilver. A sequential resilver copies the data in disk order without verifying each block checksum, restoring redundancy much sooner. Since the copy skips checksum verification, ZFS starts a [scrub](#) of the pool automatically after the sequential resilver completes. Sequential resilvering works on mirror and [dRAID](#) vdevs, not on RAID-Z. Restart an in-progress resilver from the beginning with `zpool resilver mypool`.

23.3.10. Dealing with Failed Devices

When a disk in a pool fails, the vdev to which the disk belongs enters the [degraded](#) state. The data is still available, but with reduced performance because ZFS computes missing data from the available redundancy. To restore the vdev to a fully functional state, replace the failed physical device. ZFS is then instructed to begin the [resilver](#) operation. ZFS recomputes data on the failed device from available redundancy and writes it to the replacement device. After completion, the vdev returns to [online](#) status.

If the vdev does not have any redundancy, or if devices have failed and there is not enough redundancy to compensate, the pool enters the [faulted](#) state. Unless enough devices can reconnect, the pool becomes inoperative, requiring a data restore from backups.

When replacing a failed disk, the name of the failed disk changes to the GUID of the new disk. A new device name parameter for `zpool replace` is not required if the replacement device has the same device name.

Replace a failed disk using `zpool replace`:

```
# zpool status
pool: mypool
state: DEGRADED
status: One or more devices could not be opened.  Sufficient replicas exist for
the pool to continue functioning in a degraded state.
action: Attach the missing device and online it using 'zpool online'.
see: https://openzfs.github.io/openzfs-docs/msg/ZFS-8000-2Q
config:
```

NAME	STATE	READ	WRITE	CKSUM
mypool	DEGRADED	0	0	0
mirror-0	DEGRADED	0	0	0
ada0p3	ONLINE	0	0	0
316502962686821739	UNAVAIL	0	0	0

was /dev/ada1p3

errors: No known data errors

```
# zpool replace mypool 316502962686821739 ada2p3
```

```
# zpool status
```

```

pool: mypool
state: DEGRADED
status: One or more devices is currently being resilvered.  The pool will
        continue to function, possibly in a degraded state.
action: Wait for the resilver to complete.
scan: resilver in progress since Sat Jul 11 14:52:21 2026
      781M / 781M scanned, 641M / 781M issued at 128M/s
      640M resilvered, 82.04% done, 00:00:01 to go
config:

      NAME                STATE      READ WRITE CKSUM
mypool                DEGRADED         0     0     0
  mirror-0            DEGRADED         0     0     0
    ada0p3             ONLINE          0     0     0
    replacing-1        UNAVAIL          0     0     0
      15732067398082357289 UNAVAIL          0     0     0  was /dev/ada1p3/old
      ada2p3            ONLINE          0     0     0  (resilvering)

```

```

errors: No known data errors
# zpool status
pool: mypool
state: ONLINE
scan: resilvered 781M in 00:00:17 with 0 errors on Sat Jul 11 14:52:38 2026
config:

```

NAME	STATE	READ	WRITE	CKSUM
mypool	ONLINE	0	0	0
mirror-0	ONLINE	0	0	0
ada0p3	ONLINE	0	0	0
ada2p3	ONLINE	0	0	0

```

errors: No known data errors

```

23.3.11. Hot Spares and Automatic Replacement with zfsd

Pools with redundant vdevs can register [hot spare](#) disks that stand by to replace a failed device. A spare must be at least as large as the device it replaces. Specify spares at pool creation time by listing them after the `spare` keyword, as in `zpool create mypool mirror ada0p3 ada1p3 spare ada3p3`, or add them to an existing pool with `zpool add`:

```

# zpool add mypool spare ada3p3
# zpool status
pool: mypool
state: ONLINE
config:

```

NAME	STATE	READ	WRITE	CKSUM
------	-------	------	-------	-------


```

mypool      ONLINE      0      0      0
  mirror-0  ONLINE      0      0      0
    ada0p3  ONLINE      0      0      0
    ada1p3  ONLINE      0      0      0
spares
  ada3p3    AVAIL

```

```
errors: No known data errors
```

A spare stays idle, shown as `AVAIL`, until a device fails. Activate a spare by hand with `zpool replace`, naming the failed device and the spare:

```

# zpool replace mypool ada1p3 ada3p3
# zpool status
  pool: mypool
  state: DEGRADED
status: One or more devices could not be opened.  Sufficient replicas exist for
       the pool to continue functioning in a degraded state.
action: Attach the missing device and online it using 'zpool online'.
       see: https://openzfs.github.io/openzfs-docs/msg/ZFS-8000-2Q
  scan: resilvered 781M in 00:00:21 with 0 errors on Sat Jul 11 09:14:33 2026
config:

```

NAME	STATE	READ	WRITE	CKSUM	
mypool	DEGRADED	0	0	0	
mirror-0	DEGRADED	0	0	0	
ada0p3	ONLINE	0	0	0	
spare-1	DEGRADED	0	0	0	
ada1p3	UNAVAIL	0	0	0	cannot open
ada3p3	ONLINE	0	0	0	
spares					
ada3p3	INUSE	currently in use			

```
errors: No known data errors
```

The spare replacement is temporary by design, and the pool stays in this state until an administrator resolves it. After physically replacing the failed disk and resilvering the new device with `zpool replace`, return the spare to standby with `zpool detach mypool ada3p3`. To promote the spare to a permanent member of the vdev instead, detach the failed device with `zpool detach mypool ada1p3`.

The `autoreplace` pool property, off by default, controls what happens when a new disk appears at the same physical location as a device that previously belonged to the pool. With `autoreplace` set to `on`, ZFS replaces the old device with the new disk automatically, without requiring a `zpool replace` command:





```
# zpool set autoreplace=on mypool
```

On FreeBSD, automatic reactions to failures, including honoring `autoreplace`, are performed by `zfsd(8)`, the ZFS fault management daemon in the base system. The daemon is not enabled by default. Enable and start it with:



```
# sysrc zfsd_enable="YES"
# service zfsd start
```

`zfsd(8)` listens for pool events and:

- Activates a hot spare when a device disappears from a redundant vdev or when a vdev becomes degraded or faulted.
- Marks a vdev as faulted when it produces more than 50 I/O errors or more than 8 delayed I/O operations in a 60 second period, then activates a hot spare.
- Marks a vdev as degraded when it produces more than 50 checksum errors in a 60 second period, then activates a hot spare.
- Brings a device back online when it reappears, triggering a resilver of the data written while it was missing.
- Replaces a missing device with a new disk that appears at the same physical location when the pool has `autoreplace` set to `on`.
- Returns hot spares to standby once the resilver of a permanent replacement completes.

i Note

FreeBSD uses `zfsd(8)` for fault management. The ZFS Event Daemon (ZED) found on Linux OpenZFS platforms is not part of FreeBSD.

Inspect the event stream that drives these decisions with `zpool events`. Adding `-v` shows the full payload of each event, and `-f` keeps the command running, printing new events as they arrive. ZFS also delivers these events to `devd(8)`. The rules in `/etc/devd/zfs.conf` log them to the system log and serve as a template for custom actions, such as sending mail when a pool becomes degraded.

23.3.12. Scrubbing a Pool

Routinely `scrub` pools, ideally at least once every month. The `scrub` operation is disk-intensive and will reduce performance while running. Avoid high-demand periods when scheduling `scrub` or lower its impact with the I/O scheduler tunables described in [Tuning](#). 



```
# zpool scrub mypool
# zpool status
  pool: mypool
  state: ONLINE
    scan: scrub in progress since Sat Jul 11 20:52:54 2026
          130G / 8.60T scanned at 649M/s, 116G / 8.60T issued at 580M/s
          0B repaired, 1.32% done, 04:15:41 to go
config:
```

NAME	STATE	READ	WRITE	CKSUM
mypool	ONLINE	0	0	0
raidz2-0	ONLINE	0	0	0
ada0p3	ONLINE	0	0	0
ada1p3	ONLINE	0	0	0
ada2p3	ONLINE	0	0	0
ada3p3	ONLINE	0	0	0
ada4p3	ONLINE	0	0	0
ada5p3	ONLINE	0	0	0

```
errors: No known data errors
```

Pause a running scrub with `zpool scrub -p mypool` when it interferes with other work; running `zpool scrub mypool` again resumes it from where it stopped. To cancel a scrub operation instead, run `zpool scrub -s mypool`. `zpool scrub -w mypool` waits until the scrub completes before returning, which is useful in scripts. After a scrub reports errors, `zpool scrub -e mypool` performs an *error scrub* that re-verifies only the blocks with known errors, finishing much faster than a full scrub. The error scrub requires OpenZFS 2.2, first shipped in FreeBSD 14.0.

[periodic\(8\)](#) can run scrubs automatically. To scrub every pool whose last scrub is more than 35 days old, add this line to **/etc/periodic.conf**:

```
daily_scrub_zfs_enable="YES"
```

Set `daily_scrub_zfs_default_threshold="days"` to change the number of days between scrubs. [periodic.conf\(5\)](#) also describes per-pool thresholds. Setting `daily_status_zfs_enable="YES"` adds a pool health check to the daily [periodic\(8\)](#) report.

23.3.13. Self-Healing

The checksums stored with data blocks enable the file system to *self-heal*. This feature will automatically repair data whose checksum does not match the one recorded on another device that is part of the storage pool. For example, a mirror configuration with two disks where one drive is starting to malfunction and cannot properly store the data any more. This is worse when the data was not accessed for a long time, as with long term archive storage. Traditional file systems need to run commands that check and repair the data like [fsck\(8\)](#). These commands take time,



and in severe cases, an administrator has to decide which repair operation to perform. When ZFS detects a data block with a mismatched checksum, it tries to read the data from the mirror disk. If that disk can provide the correct data, ZFS will give that to the application and correct the data on the disk with the wrong checksum. This happens without any interaction from a system administrator during normal pool operation.

The next example shows this self-healing behavior by creating a mirrored pool of partitions **/dev/ada0p1** and **/dev/ada1p1**.

```
# zpool create healer mirror /dev/ada0p1 /dev/ada1p1
# zpool status healer
pool: healer
state: ONLINE
config:

    NAME      STATE    READ WRITE CKSUM
    healer    ONLINE    0     0     0
      mirror-0 ONLINE    0     0     0
        ada0p1 ONLINE    0     0     0
        ada1p1 ONLINE    0     0     0

errors: No known data errors
# zpool list
```

NAME	SIZE	ALLOC	FREE	CKPOINT	EXPANDSZ	FRAG	CAP	DEDUP	HEALTH	ALTROOT
healer	960M	111K	960M	-	-	0%	0%	1.00x	ONLINE	-

Copy some important data to the pool to protect from data errors using the self-healing feature and create a checksum of the pool for later comparison.

```
# cp /some/important/data /healer
# zpool list
```

NAME	SIZE	ALLOC	FREE	CKPOINT	EXPANDSZ	FRAG	CAP	DEDUP	HEALTH	ALTROOT
healer	960M	67.7M	892M	-	-	0%	7%	1.00x	ONLINE	-

```
# sha1 /healer > checksum.txt
# cat checksum.txt
SHA1 (/healer) = 2753eff56d77d9a536ece6694bf0a82740344d1f
```

Simulate data corruption by writing random data to the beginning of one of the disks in the mirror. To keep ZFS from healing the data when detected, export the pool before the corruption and import it again afterwards.

Warning

This is a dangerous operation that can destroy vital data, shown here for demonstration alone. **Do not try** it during normal operation of a storage pool. Nor should this intentional corruption example run on any disk with a file system not using ZFS on another partition in it.



Do not use any other disk device names other than the ones that are part of the pool. Ensure proper backups of the pool exist and test them before running the command!

```
# zpool export healer
# dd if=/dev/random of=/dev/ada1p1 bs=1m count=200
200+0 records in
200+0 records out
209715200 bytes transferred in 62.992162 secs (3329227 bytes/sec)
# zpool import healer
```

The pool status shows that one device has experienced an error. Note that applications reading data from the pool did not receive any incorrect data. ZFS provided data from the **ada0p1** device with the correct checksums. To find the device with the wrong checksum, look for one whose **CKSUM** column contains a nonzero value.

```
# zpool status healer
pool: healer
state: ONLINE
status: One or more devices has experienced an unrecoverable error. An
       attempt was made to correct the error. Applications are unaffected.
action: Determine if the device needs to be replaced, and clear the errors
       using 'zpool clear' or replace the device with 'zpool replace'.
       see: https://openzfs.github.io/openzfs-docs/msg/ZFS-8000-9P
config:
```

NAME	STATE	READ	WRITE	CKSUM
healer	ONLINE	0	0	0
mirror-0	ONLINE	0	0	0
ada0p1	ONLINE	0	0	0
ada1p1	ONLINE	0	0	1

```
errors: No known data errors
```

ZFS detected the error and handled it by using the redundancy present in the unaffected **ada0p1** mirror device. A checksum comparison with the original one will reveal whether the pool is consistent again.

```
# sha1 /healer >> checksum.txt
# cat checksum.txt
SHA1 (/healer) = 2753eff56d77d9a536ece6694bf0a82740344d1f
SHA1 (/healer) = 2753eff56d77d9a536ece6694bf0a82740344d1f
```

Generate checksums before and after the intentional tampering while the pool data still matches. This shows how ZFS is capable of detecting and correcting any errors automatically when the checksums differ. Note this is possible with enough redundancy present in the pool. A pool consisting of a single device has no self-healing capabilities. That is also the reason why checksums are so important in ZFS; do not disable them for any reason. ZFS requires no [fsck\(8\)](#) or similar file system consistency check program to detect and correct this, and keeps the pool available while there is a problem. A scrub operation is now required to overwrite the corrupted data on **ada1p1**.

```
# zpool scrub healer
# zpool status healer
  pool: healer
  state: ONLINE

status: One or more devices has experienced an unrecoverable error.  An
        attempt was made to correct the error.  Applications are unaffected.
action: Determine if the device needs to be replaced, and clear the errors
        using 'zpool clear' or replace the device with 'zpool replace'.
       see: https://openzfs.github.io/openzfs-docs/msg/ZFS-8000-9P
scan: scrub in progress since Sat Jul 11 10:34:02 2026
      40.8M scanned at 20.4M/s, 24.9M issued at 12.4M/s, 67.0M total
      9.63M repaired, 37.16% done, 00:00:03 to go

config:

   NAME      STATE    READ WRITE CKSUM
   healer    ONLINE      0     0     0
 mirror-0    ONLINE      0     0     0
   ada0p1    ONLINE      0     0     0
   ada1p1    ONLINE      0     0    627  (repairing)

errors: No known data errors
```

The scrub operation reads data from **ada0p1** and rewrites any data with a wrong checksum on **ada1p1**, shown by the **(repairing)** output from `zpool status`. After the operation is complete, the pool status changes to:

```
# zpool status healer
  pool: healer
  state: ONLINE

status: One or more devices has experienced an unrecoverable error.  An
        attempt was made to correct the error.  Applications are unaffected.
action: Determine if the device needs to be replaced, and clear the errors
        using 'zpool clear' or replace the device with 'zpool replace'.
       see: https://openzfs.github.io/openzfs-docs/msg/ZFS-8000-9P
scan: scrub repaired 66.5M in 00:00:06 with 0 errors on Sat Jul 11 10:34:08 2026

config:

   NAME      STATE    READ WRITE CKSUM
```

```

healer      ONLINE      0      0      0
mirror-0    ONLINE      0      0      0
ada0p1      ONLINE      0      0      0
ada1p1      ONLINE      0      0  2.72K

```

```
errors: No known data errors
```

After the scrubbing operation completes with all the data synchronized from **ada0p1** to **ada1p1**, [clear](#) the error messages from the pool status by running `zpool clear`.

```

# zpool clear healer
# zpool status healer
pool: healer
state: ONLINE
scan: scrub repaired 66.5M in 00:00:06 with 0 errors on Sat Jul 11 10:34:08 2026
config:

```

NAME	STATE	READ	WRITE	CKSUM
healer	ONLINE	0	0	0
mirror-0	ONLINE	0	0	0
ada0p1	ONLINE	0	0	0
ada1p1	ONLINE	0	0	0

```
errors: No known data errors
```

The pool is now back to a fully working state, with all error counts now zero.

23.3.14. TRIM and Initialization

Solid state drives and thinly provisioned storage perform best when ZFS tells them which blocks are no longer in use. `zpool trim` passes that information to every device in the pool that supports the TRIM or UNMAP commands:

```
# zpool trim mypool
```

Trim a single device by naming it after the pool, as in `zpool trim mypool nda0p3`. `zpool trim -w` waits until trimming completes, `-c` cancels a running trim, and `-s` suspends it; running `zpool trim` again resumes a suspended trim. On devices that support it, `zpool trim --secure` requests a secure TRIM, where the device guarantees erasing the data stored on the trimmed blocks.

`zpool status -t` shows the trim progress for each device:



```
# zpool status -t mypool
pool: mypool
state: ONLINE
scan: scrub repaired 0B in 00:12:33 with 0 errors on Sat Jul 11 03:31:11 2026
config:
```

NAME	STATE	READ	WRITE	CKSUM	
mypool	ONLINE	0	0	0	
mirror-0	ONLINE	0	0	0	
nda0p3	ONLINE	0	0	0	(23% trimmed, started at Sat Jul 11 09:15:04 2026)
nda1p3	ONLINE	0	0	0	(22% trimmed, started at Sat Jul 11 09:15:04 2026)

```
errors: No known data errors
```

Setting the `autotrim` pool property to `on` makes ZFS issue small TRIM commands continuously as the pool frees space:

```
# zpool set autotrim=on mypool
```

Automatic trimming skips small freed regions and puts a constant extra load on the devices. For most systems, leaving `autotrim` off and running a full `zpool trim` on a schedule gives better results. On FreeBSD 14.1 and later, [periodic\(8\)](#) runs such a trim daily when `/etc/periodic.conf` contains `daily_trim_zfs_enable="YES"`.

`zpool initialize` is the counterpart of trimming: it writes a pattern to all unallocated space on the specified devices, or on every eligible device in the pool when given no device names. This forces the backing storage to allocate the space up front, avoiding first-write latency on thinly provisioned virtual disks, and exercises new devices before trusting them with data.

```
# zpool initialize mypool
```

As with trimming, `-c` cancels an initialization, `-s` suspends it, and `-w` waits for it to finish; `-u` removes the initialization state from the devices. See [zpool-trim\(8\)](#) and [zpool-initialize\(8\)](#) for details.

23.3.15. Pool Checkpoints

`zpool checkpoint` saves the state of the entire pool, including every dataset and the pool configuration, so that a later rewind returns the pool to the exact state it had when creating the checkpoint. Create a checkpoint before risky administrative operations such as upgrading the operating system, enabling new pool features, or reorganizing datasets:




```
# zpool checkpoint mypool
```

A pool holds at most one checkpoint at a time. The `CKPOINT` column of `zpool list` shows how much space the checkpoint consumes as the pool diverges from the saved state:

```
# zpool list mypool
```

NAME	SIZE	ALLOC	FREE	CKPOINT	EXPANDSZ	FRAG	CAP	DEDUP	HEALTH	ALTROOT
mypool	4.50T	1.42T	3.08T	221M	-	4%	31%	1.00x	ONLINE	-

To rewind, export the pool and import it again with `--rewind-to-checkpoint`:

```
# zpool export mypool
# zpool import --rewind-to-checkpoint mypool
```

Rewinding discards every change made to the pool after creating the checkpoint, including newer snapshots and property changes. Discard a checkpoint that is no longer needed to release the space it holds; `-w` waits until the discard completes:

```
# zpool checkpoint -d -w mypool
```

While a checkpoint exists, ZFS refuses operations that change the pool structure: `zpool remove`, `zpool attach`, `zpool detach`, `zpool split`, and `zpool reguid`. Space freed after creating the checkpoint is not reclaimed until the checkpoint is discarded, so a long-lived checkpoint on a busy pool can fill it up.

Note

A checkpoint is not a replacement for [snapshots](#) or backups. It protects against administrative mistakes for a short period, lives on the same disks as the pool, and rewinding discards everything written after its creation.

See [zpool-checkpoint\(8\)](#) for details.

23.3.16. Growing a Pool

The smallest device in each vdev limits the usable size of a redundant pool. Replace the smallest device with a larger device. After completing a [replace](#) or [resilver](#) operation, the pool can grow to use the capacity of the new device. For example, consider a mirror of a 1 TB drive and a 2 TB drive. The usable space is 1 TB. When replacing the 1 TB drive with another 2 TB drive, the



resilvering process copies the existing data onto the new drive. As both of the devices now have 2 TB capacity, the mirror's available space grows to 2 TB.

The pool does not use the new capacity automatically by default. Set the `autoexpand` pool property to `on` before replacing the devices and the pool grows as soon as the last smaller device leaves the vdev:

```
# zpool set autoexpand=on mypool
```

Otherwise, start expansion by using `zpool online -e` on each device. After expanding all devices, the extra space becomes available to the pool.

To grow a RAID-Z vdev by adding a disk instead of replacing the existing ones, see [RAID-Z Expansion](#).

23.3.17. Importing and Exporting Pools

Export pools before moving them to another system. ZFS unmounts all datasets, marking each device as exported but still locked to prevent use by other disks. This allows pools to be *imported* on other machines, other operating systems that support ZFS, and even different hardware architectures (with some caveats, see [zpool\(8\)](#)). When a dataset has open files, use `zpool export -f` to force exporting the pool. Use this with caution. The datasets are forcibly unmounted, potentially resulting in unexpected behavior by the applications which had open files on those datasets.

Export a pool that is not in use:

```
# zpool export mypool
```

Importing a pool automatically mounts the datasets. If this is undesired behavior, use `zpool import -N` to prevent it. `zpool import -o` sets temporary properties for this specific import. `zpool import -o altroot=` allows importing a pool with a base mount point instead of the root of the file system. If the pool was last used on a different system and was not properly exported, force the import using `zpool import -f`. `zpool import -a` imports all pools that do not appear to be in use by another system.

[zpool-import\(8\)](#) also provides options for recovering damaged pools. `zpool import -F` attempts to rewind the pool to an earlier transaction group when the most recent ones are damaged, discarding the last few seconds of writes. `zpool import -m` allows importing a pool whose dedicated log device is missing. `zpool import -o readonly=on` imports the pool read-only, preventing all writes, which is useful for forensics or when rescuing data from a failing pool. `zpool import -R /mnt` combines `altroot=/mnt` with `cachefile=none`, keeping a rescue import from disturbing the mount paths and the pool cache of the running system.



List all available pools for import:

```
# zpool import
pool: mypool
id: 9930174748043525076
state: ONLINE
action: The pool can be imported using its name or numeric identifier.
config:

    mypool    ONLINE
    ada0p1    ONLINE
```

Import the pool with an alternative root directory:

```
# zpool import -o altroot=/mnt mypool
# zfs list
```

NAME	USED	AVAIL	REFER	MOUNTPOINT
mypool	110K	47.0G	31K	/mnt/mypool

At boot, the ZFS startup scripts enabled by `zfs_enable="YES"` in `/etc/rc.conf` import every pool recorded in `/etc/zfs/zpool.cache`. Importing a pool adds it to that cache file automatically; the `cachefile` pool property controls this behavior. Pools imported with `cachefile=none` do not come back automatically after a reboot.

23.3.17.1. Importing a GELI-Encrypted Pool

The `Encrypt Disks` option of the FreeBSD installer encrypts the pool's partitions with [geli\(8\)](#), described in [Encrypting Disk Partitions](#), rather than with [ZFS native encryption](#). On the installed system the loader prompts for the passphrase and attaches the providers before mounting the root pool, but ZFS itself knows nothing about the encryption: when booting from rescue media or moving the disks to another host, `zpool import` does not find the pool until the GELI providers are attached.

Attach the encrypted **freebsd-zfs** partition of every disk in the pool, supplying the passphrase, then import the pool:

```
# geli attach ada0p4
Enter passphrase:
# zpool import -f -R /mnt zroot
```

Use `gpart show` to identify the **freebsd-zfs** partitions; with the default installer layout the encrypted partition is **p4**. Systems installed with the legacy layout, which keeps a separate unencrypted boot pool, additionally store a key file in `/boot/encryption.key` on that pool; attach those providers with `geli attach -k /path/to/encryption.key`.



23.3.18. Upgrading a Storage Pool

After upgrading FreeBSD, or if importing a pool from a system using an older version, manually upgrade the pool to make new on-disk features available. Consider whether the pool may ever need importing on an older system before upgrading. Upgrading is a one-way process. Upgrading older pools is possible, but downgrading pools with newer features is not.

Pools do not carry a single version number anymore. Instead, each on-disk format change is an individual *feature flag*, described in [zpool-features\(7\)](#). A feature is *enabled* when the pool allows its use and becomes *active* once the pool stores data that depends on it. An enabled but inactive feature does not affect compatibility. Once a feature is active, software without support for it can no longer import the pool, although pools whose active features are all "read-only compatible" still allow read-only imports.

`zpool status` reports when supported features are not enabled on a pool:

```
# zpool status mypool
pool: mypool
state: ONLINE
status: Some supported and requested features are not enabled on the pool.
       The pool can still be used, but some features are unavailable.
action: Enable all features using 'zpool upgrade'. Once this is done,
       the pool may no longer be accessible by software that does not support
       the features. See zpool-features(7) for details.
scan: scrub repaired 0B in 00:11:03 with 0 errors on Sat Jul 11 03:15:41 2026
config:

    NAME      STATE    READ WRITE CKSUM
    mypool    ONLINE      0     0     0
      mirror-0 ONLINE      0     0     0
        ada0p1 ONLINE      0     0     0
        ada1p1 ONLINE      0     0     0

errors: No known data errors
```

Running `zpool upgrade` without arguments lists every pool with disabled features:

```
# zpool upgrade
This system supports ZFS pool feature flags.

Some supported features are not enabled on the following pools. Once a
feature is enabled the pool may become incompatible with software
that does not support the feature. See zpool-features(7) for details.

Note that the pool 'compatibility' feature can be used to inhibit
feature upgrades.
```

```
POOL  FEATURE
-----
mypool
    zilsaxattr
    head_errlog
    blake3
    block_cloning
    vdev_zaps_v2
```

Use `zpool upgrade -v` to list every feature the running system supports, together with the legacy version numbers. Enable all supported features on a pool:

```
# zpool upgrade mypool
This system supports ZFS pool feature flags.

Enabled the following features on 'mypool':
    zilsaxattr
    head_errlog
    blake3
    block_cloning
    vdev_zaps_v2
```

Pools that must stay importable on systems running older ZFS software can restrict which features `zpool create` and `zpool upgrade` enable through the `compatibility` pool property:

```
# zpool create -o compatibility=openzfs-2.1-freebsd mypool mirror /dev/ada0p1 /dev/ada1p1
```

Setting the property on an existing pool with `zpool set` works as well. Each named feature set corresponds to a file in `/usr/share/zfs/compatibility.d`, and the default value `off` allows every feature. Setting the property does not disable features that are already enabled.

Warning

Update the boot code on systems that boot from a pool to support the new pool features. Use `gpart bootcode` on the partition that contains the boot code. Two types of bootcode are available, depending on the way the system boots: GPT (the most common option) and EFI (for more modern systems).

For legacy boot using GPT, use the following command:

```
# gpart bootcode -b /boot/pmbr -p /boot/gptzfsboot -i 1 ada1
```

For systems using EFI to boot, execute the following command:



```
# mount -t msdosfs /dev/ada1p1 /boot/efi
# cp /boot/loader.efi /boot/efi/efi/boot/B00TX64.efi
# cp /boot/loader.efi /boot/efi/efi/freebsd/loader.efi
```

These instructions apply to amd64 systems. Depending on the architecture, the file has a different name. Refer to [uefi\(8\)](#) to find the names of the architecture-specific files.

Apply the bootcode to all bootable disks in the pool. See [gpart\(8\)](#) for more information.

23.3.19. Displaying Recorded Pool History

ZFS records commands that change the pool, including creating datasets, changing properties, or replacing a disk. Reviewing history about a pool's creation is useful, as is checking which user performed a specific action and when. History is not kept in a log file, but is part of the pool itself. The command to review this history is aptly named `zpool history`:



```
# zpool history
History for 'tank':
2026-07-11.10:24:05 zpool create tank mirror /dev/ada0p1 /dev/ada1p1
2026-07-11.18:50:58 zfs set atime=off tank
2026-07-11.18:51:09 zfs set checksum=fletcher4 tank
2026-07-11.18:51:18 zfs create tank/backup
```

The output shows `zpool` and `zfs` commands altering the pool in some way along with a timestamp. Commands like `zfs list` are not included. When specifying no pool name, ZFS displays history of all pools.

`zpool history` can show even more information when providing the options `-i` or `-l`. `-i` displays user-initiated events as well as internally logged ZFS events.



```
# zpool history -i
History for 'tank':
2026-07-11.10:24:05 [txg:5] create pool version 5000; software version zfs-2.2.7-FreeBSD_gd75f9ee8b
2026-07-11.18:50:53 [txg:50] set tank (21) atime=0
2026-07-11.18:50:58 zfs set atime=off tank
2026-07-11.18:51:04 [txg:53] set tank (21) checksum=7
2026-07-11.18:51:09 zfs set checksum=fletcher4 tank
```



```
2026-07-11.18:51:13 [txg:55] create tank/backup (39)
2026-07-11.18:51:18 zfs create tank/backup
```

Show more details by adding `-l`. Showing history records in a long format, including information like the name of the user who issued the command and the hostname on which the change happened.

```
# zpool history -l
History for 'tank':
2026-07-11.10:24:05 zpool create tank mirror /dev/ada0p1 /dev/ada1p1 [user 0 (root) on myzfsbox:global]
2026-07-11.18:50:58 zfs set atime=off tank [user 0 (root) on myzfsbox:global]
2026-07-11.18:51:09 zfs set checksum=fletcher4 tank [user 0 (root) on myzfsbox:global]
2026-07-11.18:51:18 zfs create tank/backup [user 0 (root) on myzfsbox:global]
```

The output shows that the `root` user created the mirrored pool with disks `/dev/ada0p1` and `/dev/ada1p1`. The hostname `myzfsbox` is also shown with each command. The hostname display becomes important when exporting the pool from one system and importing on another. It's possible to distinguish the commands issued on the other system by the hostname recorded for each command.

Combine both options to `zpool history` to give the most detailed information possible for any given pool. Pool history provides valuable information when tracking down the actions performed or when needing more detailed output for debugging.

23.3.20. Performance Monitoring

A built-in monitoring system can display pool I/O statistics in real time. It shows the amount of free and used space on the pool, read and write operations performed per second, and I/O bandwidth used. By default, ZFS monitors and displays all pools in the system. Provide a pool name to limit monitoring to that pool. A basic example:

```
# zpool iostat
```

	capacity		operations		bandwidth	
pool	alloc	free	read	write	read	write
data	288G	1.53T	2	11	11.3K	57.1K

To continuously see I/O activity, specify a number as the last parameter, indicating an interval in seconds to wait between updates. The next statistic line prints after each interval. Press `Ctrl + C` to stop this continuous monitoring. Give a second number on the command line after the interval to specify the total number of statistics to display.

Display even more detailed I/O statistics with `-v`. Each device in the pool appears with a statistic line. This is useful for seeing read and write operations performed on each device, and can help

determine if any individual device is slowing down the pool. This example shows a mirrored pool with two devices:

```
# zpool iostat -v
```

pool	capacity		operations		bandwidth	
	alloc	free	read	write	read	write
data	288G	1.53T	2	12	9.23K	61.5K
mirror	288G	1.53T	2	12	9.23K	61.5K
ada1p1	-	-	0	4	5.61K	61.7K
ada2p1	-	-	1	4	5.04K	61.7K

`zpool iostat` offers several deeper views of the I/O pipeline. `-l` adds average latency columns, splitting the total wait time into disk time and time spent in the various I/O queues. `-q` shows the number of pending and active operations in each per-vdev queue. `-w` prints full latency histograms, and `-r` prints request size histograms for each leaf vdev.

The latency columns of `-l` make it easy to spot a single slow disk dragging down a whole vdev:

```
# zpool iostat -lv data
```

pool	capacity		operations		bandwidth		total_wait		disk_wait		syncq_wait		as
	alloc	free	read	write	read	write	read	write	read	write	read	write	
data	288G	1.53T	156	212	19.4M	26.6M	14ms	186ms	5ms	184ms	1us	2us	8
mirror-0	288G	1.53T	156	212	19.4M	26.6M	14ms	186ms	5ms	184ms	1us	2us	8
ada1p1	-	-	79	106	9.7M	13.3M	3ms	5ms	3ms	4ms	1us	2us	1
ada2p1	-	-	77	106	9.7M	13.3M	212ms	187ms	208ms	183ms	1us	2us	15

In this example, **ada2p1** needs hundreds of milliseconds of disk time for I/O that its mirror partner **ada1p1** completes in single-digit milliseconds, while both handle a similar number of operations. A write to the mirror only completes when the slowest member finishes, so the write latency of the whole vdev tracks **ada2p1**. High latency at normal operation counts, without read, write, or checksum errors in `zpool status`, often indicates a disk that retries internally and is a candidate for replacement.

Scripts that must wait for background pool activity to finish can block on `zpool wait`:

```
# zpool wait -t scrub,resilver mypool
```

The command returns once none of the named activities, such as `scrub`, `resilver`, `trim`, or `remove`, are in progress. See [zpool-wait\(8\)](#) for the full list of activities.



Note

`zpool status`, `zpool list`, `zpool get`, and `zfs list` accept `-j` to produce JSON output, which is more robust for scripts to parse than the human-readable tables. JSON output requires OpenZFS 2.3 or later, first available in FreeBSD 15.0.

23.3.21. Splitting a Storage Pool

ZFS can split a pool consisting of one or more mirror vdevs into two pools. Unless otherwise specified, ZFS detaches the last member of each mirror and creates a new pool containing the same data. Be sure to make a dry run of the operation with `-n` first. This displays the details of the requested operation without actually performing it. This helps confirm that the operation will do what the user intends.

23.4. `zfs` Administration

The `zfs` utility can create, destroy, and manage all existing ZFS datasets within a pool. To manage the pool itself, use `zpool`.

23.4.1. Creating and Destroying Datasets

Unlike traditional disks and volume managers, space in ZFS is *not* preallocated. With traditional file systems, after partitioning and assigning the space, there is no way to add a new file system without adding a new disk. With ZFS, creating new file systems is possible at any time. Each *dataset* has properties including features like compression, deduplication, caching, and quotas, as well as other useful properties like readonly, case sensitivity, network file sharing, and a mount point. Nesting datasets within each other is possible and child datasets will inherit properties from their ancestors. [Delegate](#), [replicate](#), [snapshot](#), [jail](#) allows administering and destroying each dataset as a unit. Creating a separate dataset for each different type or set of files has advantages. The drawbacks to having a large number of datasets are that some commands like `zfs list` will be slower, and that mounting of hundreds or even thousands of datasets will slow the FreeBSD boot process.

Create a new dataset and enable [LZ4 compression](#) on it:

```
# zfs list
NAME                                USED  AVAIL  REFER  MOUNTPOINT
mypool                             781M  93.2G   144K   none
mypool/R00T                        777M  93.2G   144K   none
mypool/R00T/default                777M  93.2G   777M   /
mypool/tmp                         176K  93.2G   176K   /tmp
mypool/usr                         616K  93.2G   144K   /usr
mypool/usr/home                   184K  93.2G   184K   /usr/home
mypool/usr/ports                  144K  93.2G   144K   /usr/ports
```



```

mypool/usr/src      144K  93.2G  144K  /usr/src
mypool/var          1.20M  93.2G  608K  /var
mypool/var/crash    148K  93.2G  148K  /var/crash
mypool/var/log      178K  93.2G  178K  /var/log
mypool/var/mail     144K  93.2G  144K  /var/mail
mypool/var/tmp      152K  93.2G  152K  /var/tmp
# zfs create -o compress=lz4 mypool/usr/mydataset
# zfs list
NAME                USED  AVAIL  REFER  MOUNTPOINT
mypool              781M  93.2G  144K   none
mypool/R00T         777M  93.2G  144K   none
mypool/R00T/default 777M  93.2G  777M   /
mypool/tmp          176K  93.2G  176K  /tmp
mypool/usr           704K  93.2G  144K  /usr
mypool/usr/home     184K  93.2G  184K  /usr/home
mypool/usr/mydataset 87.5K  93.2G  87.5K  /usr/mydataset
mypool/usr/ports    144K  93.2G  144K  /usr/ports
mypool/usr/src      144K  93.2G  144K  /usr/src
mypool/var          1.20M  93.2G  610K  /var
mypool/var/crash    148K  93.2G  148K  /var/crash
mypool/var/log      178K  93.2G  178K  /var/log
mypool/var/mail     144K  93.2G  144K  /var/mail
mypool/var/tmp      152K  93.2G  152K  /var/tmp

```

Destroying a dataset is much quicker than deleting the files on the dataset, as it does not involve scanning the files and updating the corresponding metadata.

Destroy the created dataset:

```

# zfs list
NAME                USED  AVAIL  REFER  MOUNTPOINT
mypool              880M  93.1G  144K   none
mypool/R00T         777M  93.1G  144K   none
mypool/R00T/default 777M  93.1G  777M   /
mypool/tmp          176K  93.1G  176K  /tmp
mypool/usr          101M  93.1G  144K  /usr
mypool/usr/home     184K  93.1G  184K  /usr/home
mypool/usr/mydataset 100M  93.1G  100M  /usr/mydataset
mypool/usr/ports    144K  93.1G  144K  /usr/ports
mypool/usr/src      144K  93.1G  144K  /usr/src
mypool/var          1.20M  93.1G  610K  /var
mypool/var/crash    148K  93.1G  148K  /var/crash
mypool/var/log      178K  93.1G  178K  /var/log
mypool/var/mail     144K  93.1G  144K  /var/mail
mypool/var/tmp      152K  93.1G  152K  /var/tmp
# zfs destroy mypool/usr/mydataset
# zfs list
NAME                USED  AVAIL  REFER  MOUNTPOINT

```

mypool	781M	93.2G	144K	none
mypool/R00T	777M	93.2G	144K	none
mypool/R00T/default	777M	93.2G	777M	/
mypool/tmp	176K	93.2G	176K	/tmp
mypool/usr	616K	93.2G	144K	/usr
mypool/usr/home	184K	93.2G	184K	/usr/home
mypool/usr/ports	144K	93.2G	144K	/usr/ports
mypool/usr/src	144K	93.2G	144K	/usr/src
mypool/var	1.21M	93.2G	612K	/var
mypool/var/crash	148K	93.2G	148K	/var/crash
mypool/var/log	178K	93.2G	178K	/var/log
mypool/var/mail	144K	93.2G	144K	/var/mail
mypool/var/tmp	152K	93.2G	152K	/var/tmp

In modern versions of ZFS, `zfs destroy` is asynchronous, and the free space might take minutes to appear in the pool. Use `zpool get freeing poolname` to see the `freeing` property, which shows how much space remains to reclaim in the background. Use `zpool wait -t free poolname` to wait until the background freeing completes. If there are child datasets, like [snapshots](#) or other datasets, destroying the parent is impossible. To destroy a dataset and its children, use `-r` to recursively destroy the dataset and its children. Use `-n -v` to list datasets and snapshots destroyed by this operation, without actually destroying anything. Space reclaimed by destroying snapshots is also shown.

23.4.2. Creating and Destroying Volumes

A volume is a special dataset type. Rather than mounting as a file system, expose it as a block device under **`/dev/zvol/poolname/dataset`**. This allows using the volume for other file systems, to back the disks of a virtual machine, or to make it available to other network hosts using protocols like iSCSI or HAST.

Format a volume with any file system or without a file system to store raw data. To the user, a volume appears to be a regular disk. Putting ordinary file systems on these *zvol*s provides features that ordinary disks or file systems do not have. For example, using the `compression` property on a 250 MB volume allows creation of a compressed FAT file system.

```
# zfs create -V 250m -o compression=on tank/fat32
# zfs list tank
NAME USED AVAIL REFER MOUNTPOINT
tank 258M 670M 31K /tank
# newfs_msdos -F32 /dev/zvol/tank/fat32
# mount -t msdosfs /dev/zvol/tank/fat32 /mnt
# df -h /mnt | grep fat32

Filesystem                Size Used Avail Capacity Mounted on
/dev/zvol/tank/fat32 249M  24k  249M    0%   /mnt
```

```
# mount | grep fat32
/dev/zvol/tank/fat32 on /mnt (msdosfs, local)
```

Creating a volume reserves enough space in the pool to hold the full volume size. To skip that reservation, create a *sparse volume*, also known as a thin provisioned volume, with `-s`:

```
# zfs create -s -V 250g tank/bigdisk
```



Sparse volumes allow committing more space than the pool actually has. Writes to a sparse volume can fail with `ENOSPC` when the pool runs low on space, so use sparse volumes only where the consumer of the volume tolerates write errors.

The `volmode` property controls how ZFS exposes a volume to the operating system. Setting it to `full` exposes the volume as a GEOM provider with maximal functionality, and `geom` is an alias for `full`. Setting it to `dev` exposes the volume as a plain device node, hiding any partitions it contains. Volumes with `volmode` set to `none` are not exposed outside ZFS at all, but still support snapshots, clones, and replication, making them useful as replication targets. The property defaults to `default`, which defers to the system-wide `zvol_volmode` tunable described in [zfs\(4\)](#).

Volumes work well as virtual disks for [bhyve\(8\)](#) virtual machines and as backing storage for iSCSI extents: declare the volume device as a LUN in [ctl.conf\(5\)](#) and serve it with [ctld\(8\)](#).

Warning

Do not place swap space on ZFS volumes. When the system is low on memory, ZFS itself needs memory to complete the write that would release memory, and the system can deadlock. See the upstream OpenZFS issue [openzfs/zfs#7734](#) for details. Use a dedicated swap partition instead.

Destroying a volume is much the same as destroying a regular file system dataset. The operation is nearly instantaneous, but it may take minutes to reclaim the free space in the background.

23.4.3. Renaming a Dataset

To change the name of a dataset, use `zfs rename`. To change the parent of a dataset, use this command as well. Renaming a dataset to have a different parent dataset will change the value of those properties inherited from the parent dataset. Renaming a dataset unmounts then remounts it in the new location (inherited from the new parent dataset). To prevent this behavior, use `-u`.

Rename a dataset and move it to be under a different parent dataset:





```
# zfs list
NAME                                USED  AVAIL  REFER  MOUNTPOINT
mypool                              780M  93.2G  144K   none
mypool/R00T                        777M  93.2G  144K   none
mypool/R00T/default                777M  93.2G  777M   /
mypool/tmp                         176K  93.2G  176K  /tmp
mypool/usr                         704K  93.2G  144K  /usr
mypool/usr/home                    184K  93.2G  184K  /usr/home
mypool/usr/mydataset               87.5K  93.2G  87.5K  /usr/mydataset
mypool/usr/ports                   144K  93.2G  144K  /usr/ports
mypool/usr/src                     144K  93.2G  144K  /usr/src
mypool/var                         1.21M  93.2G  614K  /var
mypool/var/crash                   148K  93.2G  148K  /var/crash
mypool/var/log                     178K  93.2G  178K  /var/log
mypool/var/mail                    144K  93.2G  144K  /var/mail
mypool/var/tmp                     152K  93.2G  152K  /var/tmp
# zfs rename mypool/usr/mydataset mypool/var/newname
# zfs list
```

```
NAME                                USED  AVAIL  REFER  MOUNTPOINT
mypool                              780M  93.2G  144K   none
mypool/R00T                        777M  93.2G  144K   none
mypool/R00T/default                777M  93.2G  777M   /
mypool/tmp                         176K  93.2G  176K  /tmp
mypool/usr                         616K  93.2G  144K  /usr
mypool/usr/home                    184K  93.2G  184K  /usr/home
mypool/usr/ports                   144K  93.2G  144K  /usr/ports
mypool/usr/src                     144K  93.2G  144K  /usr/src
mypool/var                         1.29M  93.2G  614K  /var
mypool/var/crash                   148K  93.2G  148K  /var/crash
mypool/var/log                     178K  93.2G  178K  /var/log
mypool/var/mail                    144K  93.2G  144K  /var/mail
mypool/var/newname                 87.5K  93.2G  87.5K  /var/newname
mypool/var/tmp                     152K  93.2G  152K  /var/tmp
```

Renaming snapshots uses the same command. Due to the nature of snapshots, rename cannot change their parent dataset. To rename a recursive snapshot, specify `-r`; this will also rename all snapshots with the same name in child datasets.



```
# zfs list -t snapshot
NAME                                USED  AVAIL  REFER  MOUNTPOINT
mypool/var/newname@first_snapshot    0      -  87.5K   -
# zfs rename mypool/var/newname@first_snapshot new_snapshot_name
# zfs list -t snapshot
NAME                                USED  AVAIL  REFER  MOUNTPOINT
mypool/var/newname@new_snapshot_name  0      -  87.5K   -
```



23.4.4. Setting Dataset Properties

Each ZFS dataset has properties that control its behavior. Most properties are automatically inherited from the parent dataset, but can be overridden locally. Set a property on a dataset with `zfs set property=value dataset`. Most properties have a limited set of valid values, `zfs get` will display each possible property and valid values. Using `zfs inherit` reverts most properties to their inherited values. User-defined properties are also possible. They become part of the dataset configuration and provide further information about the dataset or its contents. To distinguish these custom properties from the ones supplied as part of ZFS, use a colon (`:`) to create a custom namespace for the property.

```
# zfs set custom:costcenter=1234 tank
# zfs get custom:costcenter tank
NAME PROPERTY          VALUE SOURCE
tank custom:costcenter 1234 local
```

To remove a custom property, use `zfs inherit` with `-r`. If the custom property is not defined in any of the parent datasets, this option removes it (but the pool's history still records the change).

```
# zfs inherit -r custom:costcenter tank
# zfs get custom:costcenter tank
NAME PROPERTY          VALUE SOURCE
tank custom:costcenter - -
# zfs get all tank | grep custom:costcenter
#
```

23.4.4.1. Getting and Setting Share Properties

Two commonly used and useful dataset properties are the NFS and SMB share options. Setting these defines if and how ZFS shares datasets on the network. On FreeBSD, ZFS integrates with the NFS server in the base system: setting the `sharenfs` property exports the dataset through `mountd(8)`. ZFS writes the export entry to `/etc/zfs/exports`, which `mountd(8)` reads in addition to `/etc/exports`. Sharing datasets requires the NFS server enabled in `/etc/rc.conf`:

```
zfs_enable="YES"
nfs_server_enable="YES"
```

The value of `sharenfs` is either `on`, `off`, or a list of `exports(5)` options to apply to the export.

To get the current status of a share, enter:

```
# zfs get sharenfs mypool/usr/home
NAME PROPERTY VALUE SOURCE
```

```
mypool/usr/home sharenfs on local
```

To enable sharing of a dataset, enter:

```
# zfs set sharenfs=on mypool/usr/home
```

Set other options for sharing datasets through NFS, such as `-alldirs`, `-maproot` and `-network`. To set options on a dataset shared through NFS, enter:

```
# zfs set sharenfs="-alldirs,-maproot=root,-network=192.168.1.0/24" mypool/usr/home
```

The `sharesmb` property is not functional on FreeBSD, as the sharing library implements NFS only. To serve a dataset over SMB, install a package such as [net/samba422](#) and export the path of the mounted dataset in its configuration.

23.4.5. NFSv4 ACLs

ZFS on FreeBSD stores NFSv4-style ACLs natively; the `acltype` dataset property defaults to `nfsv4`. Every file and directory can carry an access control list alongside the traditional permission bits, giving finer-grained control than the owner/group/other model. Display and edit these lists with [getfacl\(1\)](#) and [setfacl\(1\)](#).

Display the ACL of a file:

```
% getfacl /usr/home/alice/notes.txt
# file: /usr/home/alice/notes.txt
# owner: alice
# group: alice
  owner@:rw-p--aARwCcos:-----:allow
  group@:r-----a-R-c--s:-----:allow
  everyone@:r-----a-R-c--s:-----:allow
```

Grant an additional user write access by adding an `allow` entry:

```
% setfacl -m u:bob:rwpc::allow /usr/home/alice/notes.txt
```

Two dataset properties control how ACLs interact with traditional permission handling. The `aclmode` property (`discard`, `groupmask`, `passthrough`, or `restricted`) governs what happens to the ACL when [chmod\(1\)](#) changes the permission bits of a file. With the default, `discard`, `chmod` removes all ACL entries that do not represent the new file mode. The `aclinherit` property (`discard`, `noallow`, `restricted`, `passthrough`, or `passthrough-x`)

governs which ACL entries new files and directories inherit from their parent directory. With the default, `restricted`, inherited entries lose the `write_acl` and `write_owner` permissions.

See [setfacl\(1\)](#) and [zfsprops\(7\)](#) for the full entry syntax and property descriptions.

23.4.6. Managing Snapshots

Snapshots are one of the most powerful features of ZFS. A snapshot provides a read-only, point-in-time copy of the dataset. With Copy-On-Write (COW), ZFS creates snapshots fast by preserving older versions of the data on disk. If no snapshots exist, ZFS reclaims space for future use when data is rewritten or deleted. Snapshots preserve disk space by recording just the differences between the current dataset and a previous version. Allowing snapshots on whole datasets, not on individual files or directories. A snapshot from a dataset duplicates everything contained in it. This includes the file system properties, files, directories, permissions, and so on. Snapshots use no extra space when first created, but consume space as the blocks they reference change. Recursive snapshots taken with `-r` create snapshots with the same name on the dataset and its children, providing a consistent moment-in-time snapshot of the file systems. This can be important when an application has files on related datasets or that depend upon each other. Without snapshots, a backup would have copies of the files from different points in time.

Snapshots in ZFS provide a variety of features that even other file systems with snapshot functionality lack. A typical example of snapshot use is as a quick way of backing up the current state of the file system when performing a risky action like a software installation or a system upgrade. If the action fails, rolling back to the snapshot returns the system to the same state when creating the snapshot. If the upgrade was successful, delete the snapshot to free up space. Without snapshots, a failed upgrade often requires restoring backups, which is tedious, time consuming, and may require downtime during which the system is unusable. Rolling back to snapshots is fast, even while the system is running in normal operation, with little or no downtime. The time savings are enormous with multi-terabyte storage systems considering the time required to copy the data from backup. Snapshots are not a replacement for a complete backup of a pool, but offer a quick and easy way to store a dataset copy at a specific time.

23.4.6.1. Creating Snapshots

To create snapshots, use `zfs snapshot dataset@snapshotname`. Adding `-r` creates a snapshot recursively, with the same name on all child datasets.

Create a recursive snapshot of the entire pool:

zfs list -t all

NAME	USED	AVAIL	REFER	MOUNTPPOINT
mypool	780M	93.2G	144K	none
mypool/R00T	777M	93.2G	144K	none
mypool/R00T/default	777M	93.2G	777M	/
mypool/tmp	176K	93.2G	176K	/tmp
mypool/usr	616K	93.2G	144K	/usr
mypool/usr/home	184K	93.2G	184K	/usr/home


```

mypool/usr/ports          144K  93.2G  144K  /usr/ports
mypool/usr/src            144K  93.2G  144K  /usr/src
mypool/var                1.29M  93.2G  616K  /var
mypool/var/crash          148K  93.2G  148K  /var/crash
mypool/var/log            178K  93.2G  178K  /var/log
mypool/var/mail           144K  93.2G  144K  /var/mail
mypool/var/newname        87.5K  93.2G  87.5K  /var/newname
mypool/var/newname@new_snapshot_name    0      -  87.5K  -
mypool/var/tmp            152K  93.2G  152K  /var/tmp

```

```
# zfs snapshot -r mypool@my_recursive_snapshot
```

```
# zfs list -t snapshot
```

NAME	USED	AVAIL	REFER	MOUNTPOINT
mypool@my_recursive_snapshot	0	-	144K	-
mypool/R00T@my_recursive_snapshot	0	-	144K	-
mypool/R00T/default@my_recursive_snapshot	0	-	777M	-
mypool/tmp@my_recursive_snapshot	0	-	176K	-
mypool/usr@my_recursive_snapshot	0	-	144K	-
mypool/usr/home@my_recursive_snapshot	0	-	184K	-
mypool/usr/ports@my_recursive_snapshot	0	-	144K	-
mypool/usr/src@my_recursive_snapshot	0	-	144K	-
mypool/var@my_recursive_snapshot	0	-	616K	-
mypool/var/crash@my_recursive_snapshot	0	-	148K	-
mypool/var/log@my_recursive_snapshot	0	-	178K	-
mypool/var/mail@my_recursive_snapshot	0	-	144K	-
mypool/var/newname@new_snapshot_name	0	-	87.5K	-
mypool/var/newname@my_recursive_snapshot	0	-	87.5K	-
mypool/var/tmp@my_recursive_snapshot	0	-	152K	-

Snapshots are not shown by a normal `zfs list` operation. To list snapshots, append `-t snapshot` to `zfs list`. `-t all` displays both file systems and snapshots.

Snapshots are not mounted directly, showing no path in the `MOUNTPOINT` column. ZFS does not mention available disk space in the `AVAIL` column, as snapshots are read-only after their creation. Compare the snapshot to the original dataset:

```
# zfs list -rt all mypool/usr/home
```

NAME	USED	AVAIL	REFER	MOUNTPOINT
mypool/usr/home	184K	93.2G	184K	/usr/home
mypool/usr/home@my_recursive_snapshot	0	-	184K	-

Displaying both the dataset and the snapshot together reveals how snapshots work in [COW](#) fashion. They save the changes (*delta*) made and not the complete file system contents all over again. This means that snapshots take little space when making changes. Observe space usage even more by copying a file to the dataset, then creating a second snapshot:





```
# cp /etc/passwd /var/tmp
# zfs snapshot mypool/var/tmp@after_cp
# zfs list -rt all mypool/var/tmp
```

NAME	USED	AVAIL	REFER	MOUNTPOINT
mypool/var/tmp	206K	93.2G	118K	/var/tmp
mypool/var/tmp@my_recursive_snapshot	88K	-	152K	-
mypool/var/tmp@after_cp	0	-	118K	-

The second snapshot contains the changes to the dataset after the copy operation. This yields enormous space savings. Notice that the size of the snapshot `mypool/var/tmp@my_recursive_snapshot` also changed in the `USED` column to show the changes between itself and the snapshot taken afterwards.

23.4.6.2. Comparing Snapshots

ZFS provides a built-in command to compare the differences in content between two snapshots. This is helpful with a lot of snapshots taken over time when the user wants to see how the file system has changed over time. For example, `zfs diff` lets a user find the latest snapshot that still contains a file deleted by accident. Doing this for the two snapshots created in the previous section yields this output:



```
# zfs list -rt all mypool/var/tmp
```

NAME	USED	AVAIL	REFER	MOUNTPOINT
mypool/var/tmp	206K	93.2G	118K	/var/tmp
mypool/var/tmp@my_recursive_snapshot	88K	-	152K	-
mypool/var/tmp@after_cp	0	-	118K	-

```
# zfs diff mypool/var/tmp@my_recursive_snapshot
```

M	/var/tmp/
+	/var/tmp/passwd

The command lists the changes between the specified snapshot (in this case `mypool/var/tmp@my_recursive_snapshot`) and the live file system. The first column shows the change type:

+	Adding the path or file.
-	Deleting the path or file.
M	Modifying the path or file.



R	Renaming the path or file.
---	----------------------------

Comparing the output with the table, it becomes clear that ZFS added **passwd** after creating the snapshot `mypool/var/tmp@my_recursive_snapshot`. This also resulted in a modification to the parent directory mounted at `/var/tmp`.

Comparing two snapshots is helpful when using the ZFS replication feature to transfer a dataset to a different host for backup purposes.

Compare two snapshots by providing the full dataset name and snapshot name of both datasets:



```
# cp /var/tmp/passwd /var/tmp/passwd.copy
# zfs snapshot mypool/var/tmp@diff_snapshot
# zfs diff mypool/var/tmp@my_recursive_snapshot mypool/var/tmp@diff_snapshot
M      /var/tmp/
+      /var/tmp/passwd
+      /var/tmp/passwd.copy
# zfs diff mypool/var/tmp@my_recursive_snapshot mypool/var/tmp@after_cp
M      /var/tmp/
+      /var/tmp/passwd
```

A backup administrator can compare two snapshots received from the sending host and determine the actual changes in the dataset. See the [Replication](#) section for more information.

23.4.6.3. Snapshot Rollback

When at least one snapshot is available, roll back to it at any time. Most often this is the case when the current state of the dataset is no longer valid or an older version is preferred. Scenarios such as local development tests gone wrong, botched system updates hampering the system functionality, or the need to restore deleted files or directories are all too common occurrences. To roll back a snapshot, use `zfs rollback snapshotname`. If a lot of changes are present, the operation will take a long time. During that time, the dataset always remains in a consistent state, much like a database that conforms to ACID principles is performing a rollback. This is happening while the dataset is live and accessible without requiring a downtime. Once the snapshot rolled back, the dataset has the same state as it had when the snapshot was originally taken. Rolling back to a snapshot discards all other data in that dataset not part of the snapshot. Taking a snapshot of the current state of the dataset before rolling back to a previous one is a good idea when requiring some data later. This way, the user can roll back and forth between snapshots without losing data that is still valuable.

In the first example, roll back a snapshot because a careless `rm` operation removed more data than intended.





```
# zfs list -rt all mypool/var/tmp
NAME                                USED  AVAIL  REFER  MOUNTPOINT
mypool/var/tmp                     262K  93.2G   120K   /var/tmp
mypool/var/tmp@my_recursive_snapshot  88K    -    152K   -
mypool/var/tmp@after_cp            53.5K    -    118K   -
mypool/var/tmp@diff_snapshot         0      -    120K   -
# ls /var/tmp
passwd          passwd.copy    vi.recover
# rm /var/tmp/passwd*
# ls /var/tmp
vi.recover
```

At this point, the user notices the removal of extra files and wants them back. ZFS provides an easy way to get them back using rollbacks, when performing snapshots of important data on a regular basis. To get the files back and start over from the last snapshot, issue the command:



```
# zfs rollback mypool/var/tmp@diff_snapshot
# ls /var/tmp
passwd          passwd.copy    vi.recover
```

The rollback operation restored the dataset to the state of the last snapshot. Rolling back to a snapshot taken much earlier with other snapshots taken afterwards is also possible. When trying to do this, ZFS will issue this warning:



```
# zfs list -rt snapshot mypool/var/tmp
NAME                                USED  AVAIL  REFER  MOUNTPOINT
mypool/var/tmp@my_recursive_snapshot  88K    -    152K   -
mypool/var/tmp@after_cp            53.5K    -    118K   -
mypool/var/tmp@diff_snapshot         0      -    120K   -
# zfs rollback mypool/var/tmp@my_recursive_snapshot
cannot rollback to 'mypool/var/tmp@my_recursive_snapshot': more recent snapshots exist
use '-r' to force deletion of the following snapshots:
mypool/var/tmp@after_cp
mypool/var/tmp@diff_snapshot
```

This warning means that snapshots exist between the current state of the dataset and the snapshot to which the user wants to roll back. To complete the rollback delete these snapshots. ZFS cannot track all the changes between different states of the dataset, because snapshots are read-only. ZFS will not delete the affected snapshots unless the user specifies `-r` to confirm that this is the desired action. If that is the intention, and understanding the consequences of losing all intermediate snapshots, issue the command:



```
# zfs rollback -r mypool/var/tmp@my_recursive_snapshot
# zfs list -rt snapshot mypool/var/tmp
```

NAME	USED	AVAIL	REFER	MOUNTPOINT
mypool/var/tmp@my_recursive_snapshot	8K	-	152K	-

```
# ls /var/tmp
vi.recover
```

The output from `zfs list -t snapshot` confirms the removal of the intermediate snapshots as a result of `zfs rollback -r`.

23.4.6.4. Restoring Individual Files from Snapshots

Snapshots live in a hidden directory under the parent dataset: **.zfs/snapshot/snapshotname**. By default, these directories will not show even when executing a standard `ls -a`. Although the directory doesn't show, access it like any normal directory. The property named `snapdir` controls whether these hidden directories show up in a directory listing. Setting the property to `visible` allows them to appear in the output of `ls` and other commands that deal with directory contents.

```
# zfs get snapdir mypool/var/tmp
```

NAME	PROPERTY	VALUE	SOURCE
mypool/var/tmp	snapdir	hidden	default

```
# ls -a /var/tmp
.          ..          passwd          vi.recover
# zfs set snapdir=visible mypool/var/tmp
# ls -a /var/tmp
.          ..          .zfs          passwd          vi.recover
```

Restore individual files to a previous state by copying them from the snapshot back to the parent dataset. The directory structure below **.zfs/snapshot** has a directory named like the snapshots taken earlier to make it easier to identify them. The next example shows how to restore a file from the hidden **.zfs** directory by copying it from the snapshot containing the latest version of the file:

```
# rm /var/tmp/passwd
# ls -a /var/tmp
.          ..          .zfs          vi.recover
# ls /var/tmp/.zfs/snapshot
after_cp          my_recursive_snapshot
# ls /var/tmp/.zfs/snapshot/after_cp
passwd          vi.recover
# cp /var/tmp/.zfs/snapshot/after_cp/passwd /var/tmp
```

Even if the `snapdir` property is set to hidden, running `ls .zfs/snapshot` will still list the contents of that directory. The administrator decides whether to display these directories. This is a

per-dataset setting. Copying files or directories from this hidden **.zfs/snapshot** is simple enough. Trying it the other way around results in this error:

```
# cp /etc/rc.conf /var/tmp/.zfs/snapshot/after_cp/  
cp: /var/tmp/.zfs/snapshot/after_cp/rc.conf: Read-only file system
```

The error reminds the user that snapshots are read-only and cannot change after creation. Copying files into and removing them from snapshot directories are both disallowed because that would change the state of the dataset they represent.

Snapshots consume space based on how much the parent file system has changed since the time of the snapshot. The **used** property of a snapshot tracks the space the snapshot uses; its **written** property shows how much data was written between the previous snapshot and this one.

To destroy snapshots and reclaim the space, use **zfs destroy dataset@snapshot**. Adding **-r** recursively removes all snapshots with the same name under the parent dataset. Adding **-n -v** to the command displays a list of the snapshots to be deleted and an estimate of the space it would reclaim without performing the actual destroy operation.

23.4.6.5. Snapshot Holds

A **hold** places a user-defined tag on a snapshot and prevents its destruction. Attempts to destroy a held snapshot with **zfs destroy** fail with **EBUSY**. A snapshot can carry any number of holds, each identified by a unique tag name. Holds protect snapshots that other processes still depend on, such as the most recent snapshot shared by both sides of a replication chain (see [Replication](#)). Destroying that common snapshot by accident forces the next replication to start over with a full stream.

Place a hold with **zfs hold tag snapshot**. Adding **-r** also holds the snapshots with the same name on all child datasets.

```
# zfs hold keepme mypool/var/tmp@my_recursive_snapshot  
# zfs destroy mypool/var/tmp@my_recursive_snapshot  
cannot destroy snapshot mypool/var/tmp@my_recursive_snapshot: dataset is busy
```

zfs holds lists the holds on a snapshot, showing the tag name and the time of placing each hold:

```
# zfs holds mypool/var/tmp@my_recursive_snapshot  
NAME                                TAG      TIMESTAMP  
mypool/var/tmp@my_recursive_snapshot keepme    Sat Jul 11 09:41 2026
```



Remove a hold with `zfs release`. Destroying the snapshot becomes possible again once the last hold on it is released:

```
# zfs release keepme mypool/var/tmp@my_recursive_snapshot
# zfs destroy mypool/var/tmp@my_recursive_snapshot
```

See [zfs-hold\(8\)](#) for more information.

23.4.7. Bookmarks

A [bookmark](#) records the point in time at which a snapshot was created, without keeping any of the snapshot data. Bookmarks consume almost no space, no matter how much the dataset changes afterwards. Like a snapshot, a bookmark can serve as the source of an incremental `zfs send`. Unlike a snapshot, it does not prevent ZFS from freeing the old data. This combination makes bookmarks useful for incremental replication: after sending a snapshot to another pool, bookmark it and destroy it on the sending side, reclaiming the space while the receiving side keeps its copy of the snapshot. See [Incremental Backups](#) for the replication workflow itself.

Create a bookmark from a snapshot with `zfs bookmark`. Bookmark names use `#` as the separator between the dataset name and the bookmark name, in the same way snapshot names use `@`.

```
# zfs snapshot mypool/data@snap1
# zfs send mypool/data@snap1 | zfs receive backup/data
# zfs bookmark mypool/data@snap1 mypool/data#snap1bm
# zfs destroy mypool/data@snap1
```

List bookmarks with `zfs list -t bookmark`:

```
# zfs list -t bookmark mypool/data
```

NAME	USED	AVAIL	REFER	MOUNTPOINT
mypool/data#snap1bm	-	-	184K	-

When the time comes for the next incremental backup, take a new snapshot and use the bookmark as the incremental source:

```
# zfs snapshot mypool/data@snap2
# zfs send -i mypool/data#snap1bm mypool/data@snap2 | zfs receive backup/data
```

This works because the receiving pool still has the snapshot *backup/data@snap1* matching the bookmark. Bookmarks serve as incremental sources only. Mounting a bookmark, rolling back to it, 

or restoring files from it is impossible, as it contains no data. See [zfs-bookmark\(8\)](#) for more information.

23.4.8. Managing Clones

A clone is a copy of a snapshot treated more like a regular dataset. Unlike a snapshot, a clone is writeable and mountable, and has its own properties. After creating a clone using `zfs clone`, destroying the originating snapshot is impossible. To reverse the child/parent relationship between the clone and the snapshot use `zfs promote`. Promoting a clone makes the snapshot become a child of the clone, rather than of the original parent dataset. This will change how ZFS accounts for the space, but not actually change the amount of space consumed. Mounting the clone anywhere within the ZFS file system hierarchy is possible, not only below the original location of the snapshot.

To show the clone feature use this example dataset:

```
# zfs list -rt all camino/home/joe
```

NAME	USED	AVAIL	REFER	MOUNTPOINT
camino/home/joe	108K	1.3G	87K	/usr/home/joe
camino/home/joe@plans	21K	-	85.5K	-
camino/home/joe@backup	0K	-	87K	-

A typical use for clones is to experiment with a specific dataset while keeping the snapshot around to fall back to in case something goes wrong. Since snapshots cannot change, create a read/write clone of a snapshot. After achieving the desired result in the clone, promote the clone to a dataset and remove the old file system. Removing the parent dataset is not strictly necessary, as the clone and dataset can coexist without problems.

```
# zfs clone camino/home/joe@backup camino/home/joenew
# ls /usr/home/joe*
/usr/home/joe:
backup.txz    plans.txt

/usr/home/joenew:
backup.txz    plans.txt
# df -h /usr/home
```

Filesystem	Size	Used	Avail	Capacity	Mounted on
usr/home/joe	1.3G	31k	1.3G	0%	/usr/home/joe
usr/home/joenew	1.3G	31k	1.3G	0%	/usr/home/joenew

Creating a clone makes it an exact copy of the state the dataset was in when taking the snapshot. Changing the clone independently from its originating dataset is possible now. The connection between the two is the snapshot. ZFS records this connection in the property `origin`. Promoting the clone with `zfs promote` makes the clone an independent dataset. This removes the value

the `origin` property and disconnects the newly independent dataset from the snapshot. This example shows it:

```
# zfs get origin camino/home/joenew
NAME                                PROPERTY  VALUE                                SOURCE
camino/home/joenew                 origin    camino/home/joe@backup              -
# zfs promote camino/home/joenew
# zfs get origin camino/home/joenew
NAME                                PROPERTY  VALUE                                SOURCE
camino/home/joenew                 origin    -                                    -
```

After making some changes like copying **loader.conf** to the promoted clone, for example, the old directory becomes obsolete in this case. Instead, the promoted clone can replace it. To do this, `zfs destroy` the old dataset first and then `zfs rename` the clone to the old dataset name (or to an entirely different name).

```
# cp /boot/defaults/loader.conf /usr/home/joenew
# zfs destroy -f camino/home/joe
# zfs rename camino/home/joenew camino/home/joe
# ls /usr/home/joe
backup.txz      loader.conf     plans.txt
# df -h /usr/home
Filesystem      Size    Used    Avail Capacity  Mounted on
usr/home/joe    1.3G    128k    1.3G      0%      /usr/home/joe
```

The cloned snapshot is now an ordinary dataset. It contains all the data from the original snapshot plus the files added to it like **loader.conf**. Clones provide useful features to ZFS users in different scenarios. For example, provide jails as snapshots containing different sets of installed applications. Users can clone these snapshots and add their own applications as they see fit. Once satisfied with the changes, promote the clones to full datasets and provide them to end users to work with like they would with a real dataset. This saves time and administrative overhead when providing these jails.

23.4.9. Block Cloning

[Block cloning](#) brings copy-on-write to individual file copies. When enabled, copying a file with `cp(1)` does not duplicate the file data. Instead, the new file refers to the same blocks as the original, and ZFS writes new blocks only when either file changes afterwards. `cp(1)` performs copies with `copy_file_range(2)`, which ZFS turns into block clone operations, making copies within a pool almost instantaneous and consuming no additional space at first.

Unlike [deduplication](#), block cloning keeps no memory-hungry table of checksums. Sharing happens at the moment of the copy only, and writing identical data in separate operations still



stores it twice. Unlike [clones](#), block cloning requires no snapshot, works on individual files rather than whole datasets, and leaves no `origin` relationship behind.

Block cloning requires the `block_cloning` pool feature from OpenZFS 2.2, first shipped in FreeBSD 14.0 (see [zpool-features\(7\)](#)). The `sysctl(8)` `vfs.zfs.bclone_enabled` controls whether ZFS actually creates block clones. It defaults to 0 on FreeBSD 14.x and to 1 on FreeBSD 15.0. While the `sysctl` is 0, `copy_file_range()` falls back to an ordinary copy. To enable block cloning on FreeBSD 14.x:

```
# sysctl vfs.zfs.bclone_enabled=1
vfs.zfs.bclone_enabled: 0 -> 1
```

To keep the setting across reboots, add this line to **`/etc/sysctl.conf`**:

```
vfs.zfs.bclone_enabled=1
```

The pool properties `bcloneused`, `bclonesaved`, and `bcloneratio` show the space used by cloned blocks, the space saved by cloning, and the resulting savings ratio:

```
# cp /var/tmp/database.dump /var/tmp/database.copy
# zpool list -o name,bcloneused,bclonesaved,bcloneratio mypool
```

NAME	BCLONE_USED	BCLONE_SAVED	BCLONE_RATIO
mypool	1.7G	1.7G	2.00x

23.4.10. Replication

Keeping data on a single pool in one location exposes it to risks like theft and natural or human disasters. Making regular backups of the entire pool is vital. ZFS provides a built-in serialization feature that can send a stream representation of the data to standard output. Using this feature, storing this data on another pool connected to the local system is possible, as is sending it over a network to another system. Snapshots are the basis for this replication (see the section on [ZFS snapshots](#)). The commands used for replicating data are `zfs send` and `zfs receive`.

These examples show ZFS replication with these two pools:

```
# zpool list
```

NAME	SIZE	ALLOC	FREE	CKPOINT	EXPANDSZ	FRAG	CAP	DEDUP	HEALTH	ALTROOT
backup	960M	77K	896M	-	-	0%	0%	1.00x	ONLINE	-
mypool	984M	43.7M	940M	-	-	0%	4%	1.00x	ONLINE	-

The pool named *mypool* is the primary pool where writing and reading data happens on a regular basis. Using a second standby pool *backup* in case the primary pool becomes unavailable. Note 

that this fail-over is not done automatically by ZFS, but must be manually done by a system administrator when needed. Use a snapshot to provide a consistent file system version to replicate. After creating a snapshot of *mypool*, copy it to the *backup* pool by replicating snapshots. This does not include changes made since the most recent snapshot.

```
# zfs snapshot mypool@backup1
# zfs list -t snapshot
```

NAME	USED	AVAIL	REFER	MOUNTPOINT
mypool@backup1	0	-	43.6M	-

Now that a snapshot exists, use `zfs send` to create a stream representing the contents of the snapshot. Store this stream as a file or receive it on another pool. Write the stream to standard output, but redirect to a file or pipe or an error appears:

```
# zfs send mypool@backup1
Error: Stream can not be written to a terminal.
You must redirect standard output.
```

To back up a dataset with `zfs send`, redirect to a file located on the mounted backup pool. Ensure that the pool has enough free space to accommodate the size of the sent snapshot, which means the data contained in the snapshot, not the changes from the previous snapshot.

```
# zfs send mypool@backup1 > /backup/backup1
# zpool list
```

NAME	SIZE	ALLOC	FREE	CKPOINT	EXPANDSZ	FRAG	CAP	DEDUP	HEALTH	ALTROOT
backup	960M	63.7M	896M	-	-	0%	6%	1.00x	ONLINE	-
mypool	984M	43.7M	940M	-	-	0%	4%	1.00x	ONLINE	-

The `zfs send` transferred all the data in the snapshot called *backup1* to the pool named *backup*. To create and send these snapshots automatically, use a [cron\(8\)](#) job.

Instead of storing the backups as archive files, ZFS can receive them as a live file system, allowing direct access to the backed up data. To get to the actual data contained in those streams, use `zfs receive` to transform the streams back into files and directories. The example below combines `zfs send` and `zfs receive` using a pipe to copy the data from one pool to another. Use the data directly on the receiving pool after the transfer is complete. It is only possible to replicate a dataset to an empty dataset.

```
# zfs snapshot mypool@replica1
# zfs send -v mypool@replica1 | zfs receive backup/mypool
full send of mypool@replica1 estimated size is 50.1M
total estimated size is 50.1M
TIME      SENT      SNAPSHOT mypool@replica1
```

```
10:22:01 9.75M mypool@replica1
10:22:02 25.4M mypool@replica1
```

```
# zpool list
```

NAME	SIZE	ALLOC	FREE	CKPOINT	EXPANDSZ	FRAG	CAP	DEDUP	HEALTH	ALTROOT
backup	960M	63.7M	896M	-	-	0%	6%	1.00x	ONLINE	-
mypool	984M	43.7M	940M	-	-	0%	4%	1.00x	ONLINE	-

23.4.10.1. Incremental Backups

`zfs send` can also determine the difference between two snapshots and send individual differences between the two. This saves disk space and transfer time. For example:

```
# zfs snapshot mypool@replica2
```

```
# zfs list -t snapshot
```

NAME	USED	AVAIL	REFER	MOUNTPOINT
mypool@replica1	5.72M	-	43.6M	-
mypool@replica2	0	-	44.1M	-

```
# zpool list
```

NAME	SIZE	ALLOC	FREE	CKPOINT	EXPANDSZ	FRAG	CAP	DEDUP	HEALTH	ALTROOT
backup	960M	61.7M	898M	-	-	0%	6%	1.00x	ONLINE	-
mypool	960M	50.2M	910M	-	-	0%	5%	1.00x	ONLINE	-

Create a second snapshot called *replica2*. This second snapshot contains changes made to the file system between now and the previous snapshot, *replica1*. Using `zfs send -i` and indicating the pair of snapshots generates an incremental replica stream containing the changed data. This succeeds if the initial snapshot already exists on the receiving side.

```
# zfs send -v -i mypool@replica1 mypool@replica2 | zfs receive backup/mypool
```

```
send from @replica1 to mypool@replica2 estimated size is 5.02M
```

```
total estimated size is 5.02M
```

```
TIME SENT SNAPSHOT mypool@replica2
```

```
# zpool list
```

NAME	SIZE	ALLOC	FREE	CKPOINT	EXPANDSZ	FRAG	CAP	DEDUP	HEALTH	ALTROOT
backup	960M	80.8M	879M	-	-	0%	8%	1.00x	ONLINE	-
mypool	960M	50.2M	910M	-	-	0%	5%	1.00x	ONLINE	-

```
# zfs list
```

NAME	USED	AVAIL	REFER	MOUNTPOINT
backup	55.4M	240G	152K	/backup
backup/mypool	55.3M	240G	55.2M	/backup/mypool
mypool	55.6M	11.6G	55.0M	/mypool

```
# zfs list -t snapshot
```

NAME	USED	AVAIL	REFER	MOUNTPOINT
------	------	-------	-------	------------

backup/mypool@replica1	104K	-	50.2M	-
backup/mypool@replica2	0	-	55.2M	-
mypool@replica1	29.9K	-	50.0M	-
mypool@replica2	0	-	55.0M	-

The incremental stream replicated the changed data rather than the entirety of *replica1*. Sending the differences alone took much less time to transfer and saved disk space by not copying the whole pool each time. This is useful when replicating over a slow network or one charging per transferred byte.

A new file system, *backup/mypool*, is available with the files and data from the pool *mypool*. Specifying `-p` copies the dataset properties including compression settings, quotas, and mount points. Specifying `-R` copies all child datasets of the dataset along with their properties. Automate sending and receiving to create regular backups on the second pool. The [Send Stream Options](#) section describes these and other useful options in more detail.

23.4.10.2. Send Stream Options

`zfs send` provides options that control what a stream includes and how compact it is. Specifying `-p` includes the dataset properties, such as compression settings, quotas, and mount points, in the stream. Specifying `-R` generates a replication stream package, which includes the dataset, all child datasets, snapshots, clones, and properties up to the named snapshot, and implies `-p`. The example in [Sending Encrypted Backups over SSH](#) uses `-R` to replicate a complete dataset tree. Specifying `-v` prints information about the generated stream, including a per-second progress report.

Other options change the format of the data blocks in the stream:

- `-L` permits blocks larger than 128 KB in the stream, preserving the on-disk block size of datasets with a `recordsize` above 128 KB instead of splitting those blocks.
- `-e` generates a more compact stream by using `WRITE_EMBEDDED` records for tiny blocks that the `embedded_data` pool feature stores directly in the block pointer.
- `-c` sends blocks that are compressed on disk in their compressed form rather than decompressing them first, saving CPU time on both systems and reducing the amount of data transferred for compressed datasets.
- `-w` sends an encrypted dataset in raw form, exactly as stored on disk and without loading its encryption key, as described in [Sending Encrypted Datasets](#). For unencrypted datasets, `-w` is equivalent to `-Lec`.

Combining these options as `zfs send -Lec` is a good default on modern pools. The receiving pool must support every feature the stream uses, and `zfs receive` rejects a stream that requires a feature the pool lacks. See [zfs-send\(8\)](#) and [zpool-features\(7\)](#) for the full list of options and the pool features they depend on.

23.4.10.3. Resumable Transfers



A dropped network connection or a reboot during a large transfer normally means starting over from the beginning. Using `zfs receive -s` keeps the partially received state on the receiving dataset instead of deleting it when the stream ends prematurely.

```
# zfs send -v mypool@replica1 | zfs receive -s backup/mypool
full send of mypool@replica1 estimated size is 50.1M
total estimated size is 50.1M
TIME          SENT    SNAPSHOT mypool@replica1
10:31:01      9.75M    mypool@replica1
^C
```

After the interruption, the receiving dataset stores an opaque resume token in its `receive_resume_token` property. The token records which snapshot the interrupted stream contained and how much of it arrived. Passing the token to `zfs send -t` generates a new stream that continues where the transfer stopped:

```
# zfs get -H -o value receive_resume_token backup/mypool
1-1211c4f4a-f8-789c636064000310a501c49c50360710a715e5e7a69766a63040c1eabb735735ce8f8d5420c0e5c9e8d4
# zfs send -t $(zfs get -H -o value receive_resume_token backup/mypool) | zfs receive -s backup/mypool
```

The resumed transfer completes normally and removes the saved partial state. Until resumed or aborted, the partial state consumes space on the receiving pool and blocks other receives into the same dataset. To abandon an interrupted transfer instead of resuming it, use `zfs receive -A` to delete the saved partial state:

```
# zfs receive -A backup/mypool
```

23.4.10.4. Sending Encrypted Backups over SSH

Sending streams over the network is a good way to keep a remote backup, but it does come with a drawback. Data sent over the network link is not encrypted, allowing anyone to intercept and transform the streams back into data without the knowledge of the sending user. This is undesirable when sending the streams over the internet to a remote host. Use SSH to securely encrypt data sent over a network connection. Since ZFS requires redirecting the stream from standard output, piping it through SSH is easy. To keep the contents of the file system encrypted on the remote system as well, use [native encryption](#) and send the dataset as a raw stream, as described in [Sending Encrypted Datasets](#).

Change some settings and take security precautions first. This describes the necessary steps required for the `zfs send` operation; for more information on SSH, see [OpenSSH](#).

Change the configuration as follows:

- Passwordless SSH access between sending and receiving host using SSH keys



- ZFS requires the privileges of the `root` user to send and receive streams. This requires logging in to the receiving system as `root`.
- Security reasons prevent `root` from logging in by default.
- Use the [ZFS Delegation](#) system to allow a non-`root` user on each system to perform the respective send and receive operations. On the sending system:

```
# zfs allow -u someuser send,snapshot mypool
```

- To mount the pool, the unprivileged user must own the directory, and regular users need permission to mount file systems.

On the receiving system:

```
# sysctl vfs.usermount=1
vfs.usermount: 0 -> 1
# echo vfs.usermount=1 >> /etc/sysctl.conf
# zfs create recvpool/backup
# zfs allow -u someuser create,mount,receive recvpool/backup
# chown someuser /recvpool/backup
```

The unprivileged user can receive and mount datasets now, and replicates the *home* dataset to the remote system:

```
% zfs snapshot -r mypool/home@monday
% zfs send -R mypool/home@monday | ssh someuser@backuphost zfs recv -dvu recvpool/backup
```

Create a recursive snapshot called *monday* of the file system dataset *home* on the pool *mypool*. Then `zfs send -R` includes the dataset, all child datasets, snapshots, clones, and settings in the stream. Pipe the output through SSH to the waiting `zfs receive` on the remote host *backuphost*. Using an IP address or fully qualified domain name is good practice. The receiving machine writes the data to the *backup* dataset on the *recvpool* pool. Adding `-d` to `zfs recv` discards the first element of the sent snapshot's path, usually the pool name, and grafts the remaining path onto the receiving dataset, creating any required intermediate file systems. `-u` causes the file systems to not mount on the receiving side. Using `-v` shows more details about the transfer, including the elapsed time and the amount of data transferred.

23.4.10.5. Corrective Receive

A scrub detects corrupted data, but on a pool without enough redundancy it cannot repair the damage and reports permanent errors instead. When an intact copy of the affected snapshot exists on another pool, a corrective receive with `zfs receive -c` heals the damaged blocks i

place, using the send stream as a source of healthy data. The dataset stays where it is, without any rollback or rename.

Note

Corrective receive requires OpenZFS 2.2, first shipped in FreeBSD 14.0.

In this example, a scrub of the single-disk pool *mypool* found unrecoverable damage:

```
# zpool status -v mypool
pool: mypool
state: ONLINE
status: One or more devices has experienced an error resulting in data
corruption. Applications may be affected.
action: Restore the file in question if possible. Otherwise restore the
entire pool from backup.
see: https://openzfs.github.io/openzfs-docs/msg/ZFS-8000-8A
scan: scrub repaired 0B in 00:01:22 with 1 errors on Sat Jul 11 09:14:37 2026
config:

    NAME      STATE    READ WRITE CKSUM
    mypool    ONLINE   0     0     0
    ada0      ONLINE   0     0     2

errors: Permanent errors have been detected in the following files:

    /mypool/reports/summary-2026.db
```

The snapshot *replica2* still references the damaged blocks, and the *backup* pool holds an intact copy of that snapshot. Send the snapshot from the backup pool and receive it correctively into the matching snapshot on the damaged pool, then scrub to confirm the repair:

```
# zfs send backup/mypool@replica2 | zfs receive -c mypool@replica2
# zpool scrub -w mypool
# zpool status mypool | grep errors:
errors: No known data errors
```

The stream must contain the same snapshot that exists on the damaged dataset. Corrective receive heals only the data blocks present in the stream; it cannot repair metadata or data written after the snapshot was taken. Always run a [scrub](#) afterwards to verify that the pool no longer contains damaged data.



Use [Dataset quotas](#) to restrict the amount of space consumed by a particular dataset. [Reference Quotas](#) work in much the same way, but count the space used by the dataset itself, excluding snapshots and child datasets. Similarly, use [user](#) and [group](#) quotas to prevent users or groups from using up all the space in the pool or dataset. Project quotas, described below, restrict the space consumed by an arbitrary directory tree instead.

The following examples assume that the users already exist in the system. Before adding a user to the system, make sure to create their home dataset first and set the `mountpoint` to `/home/bob`. Then, create the user and make the home directory point to the dataset's `mountpoint` location. This will properly set owner and group permissions without shadowing any pre-existing home directory paths that might exist.

To enforce a dataset quota of 10 GB for **storage/home/bob**:

```
# zfs set quota=10G storage/home/bob
```



To enforce a reference quota of 10 GB for **storage/home/bob**:

```
# zfs set refquota=10G storage/home/bob
```



To remove a quota of 10 GB for **storage/home/bob**:

```
# zfs set quota=none storage/home/bob
```



The general format is `userquota@user=size`, and the user's name must be in one of these formats:

- POSIX compatible name such as *joe*.
- POSIX numeric ID such as *789*.
- SID name such as *joe.bloggs@example.com*.
- SID numeric ID such as *S-1-123-456-789*.

For example, to enforce a user quota of 50 GB for the user named *joe*:

```
# zfs set userquota@joe=50G storage/home/joe
```



To remove any quota:

```
# zfs set userquota@joe=none storage/home/joe
```



Note

User quota properties are not displayed by `zfs get all`. Non-`root` users can't see other's quotas unless granted the `userquota` privilege. Users with this privilege are able to view and set everyone's quota.

The general format for setting a group quota is: `groupquota@group=size`.

To set the quota for the group *firstgroup* to 50 GB, use:

```
# zfs set groupquota@firstgroup=50G storage/home
```



To remove the quota for the group *firstgroup*, or to make sure that one is not set, instead use:

```
# zfs set groupquota@firstgroup=none storage/home
```



As with the user quota property, non-`root` users can see the quotas associated with the groups to which they belong. A user with the `groupquota` privilege or `root` can view and set all quotas for all groups.

To display the amount of space used by each user on a file system or snapshot along with any quotas, use `zfs userspace`. For group information, use `zfs groupspace`. For more information about supported options or how to display specific options alone, refer to [zfs-userspace\(8\)](#).

Privileged users and `root` can list the quota for **storage/home/bob** using:

```
# zfs get quota storage/home/bob
```



23.4.11.1. Project Quotas

Dataset and user quotas do not fit every layout. When several directory trees belonging to different projects share a single dataset, project quotas restrict the space consumed by each tree instead. A project is an arbitrary numeric identifier tagged onto files and directories.

`zfs project` manages the project ID of files and directories. Setting the project inherit flag on a directory makes new files and subdirectories created inside it inherit its project ID. To assign project ID 42 to an existing tree, set the flag and the ID recursively:

```
# zfs project -s -p 42 -r /storage/proj
```



Enforce a limit of 100 GB on all files belonging to project 42 with the `projectquota` property:



```
# zfs set projectquota@42=100G storage
```



`zfs projectspace` displays the space consumed by each project on a dataset along with any quotas:

```
# zfs projectspace -o name,used,quota storage
```

NAME	USED	QUOTA
42	1.95G	100G



The `projectobjquota` property limits the number of objects a project may own in the same way. Display or clear the project ID of a directory with `zfs project -d` and `zfs project -C`. Refer to [zfs-project\(8\)](#) and [zfsprops\(7\)](#) for details.

23.4.12. Reservations

[Reservations](#) guarantee an always-available amount of space on a dataset. The reserved space will not be available to any other dataset. This useful feature ensures that free space is available for an important dataset or log files.

The general format of the `reservation` property is `reservation=size`, so to set a reservation of 10 GB on **storage/home/bob**, use:

```
# zfs set reservation=10G storage/home/bob
```



To clear any reservation:

```
# zfs set reservation=none storage/home/bob
```



The same principle applies to the `refreservation` property for setting a [Reference Reservation](#), with the general format `refreservation=size`.

This command shows any reservations or refreservations that exist on **storage/home/bob**:

```
# zfs get reservation storage/home/bob
# zfs get refreservation storage/home/bob
```



23.4.13. Compression



ZFS provides transparent compression. Compressing data written at the block level saves space and also increases disk throughput. If data compresses by 25% the compressed data writes to the disk at the same rate as the uncompressed version, resulting in an effective write speed of 125%. Compression can also be a great alternative to [Deduplication](#) because it does not require extra memory.

ZFS offers different compression algorithms, each with different trade-offs. On OpenZFS 2.2 and later, newly created pools and datasets default to `compression=on`, which selects LZ4. LZ4 compresses the entire pool without the large performance trade-off of other algorithms. The biggest advantage to LZ4 is the *early abort* feature. If LZ4 does not achieve at least 12.5% compression in the header part of the data, ZFS writes the block uncompressed to avoid wasting CPU cycles trying to compress data that is either already compressed or uncompressible. Changing the `compression` property affects new writes only; existing data keeps its old encoding until rewritten (see [Rewriting Existing Data](#)). For details about the different compression algorithms available in ZFS, see the [Compression](#) entry in the terminology section.

The administrator can see the effectiveness of compression using dataset properties.



```
# zfs get used,compressratio,compression,logicalused mypool/compressed_dataset
```

NAME	PROPERTY	VALUE	SOURCE
mypool/compressed_dataset	used	449G	-
mypool/compressed_dataset	compressratio	1.11x	-
mypool/compressed_dataset	compression	lz4	local
mypool/compressed_dataset	logicalused	496G	-

The dataset is using 449 GB of space (the `used` property). Without compression, it would have taken 496 GB of space (the `logicalused` property). This results in a 1.11:1 compression ratio.

Compression can have an unexpected side effect when combined with [User Quotas](#). User quotas restrict how much actual space a user consumes on a dataset *after compression*. If a user has a quota of 10 GB, and writes 10 GB of compressible data, they will still be able to store more data. If they later update a file, say a database, with more or less compressible data, the amount of space available to them will change. This can result in the odd situation where a user did not increase the actual amount of data (the `logicalused` property), but the change in compression caused them to reach their quota limit.

Compression can have a similar unexpected interaction with backups. Quotas are often used to limit data storage to ensure there is enough backup space available. Since quotas do not consider compression ZFS may write more data than would fit with uncompressed backups.

23.4.14. Zstandard Compression

OpenZFS 2.0 added the Zstandard (Zstd) compression algorithm. Zstd offers higher compression ratios than the default LZ4 while offering much greater speeds than the alternative, gzip.



Zstd provides a large selection of compression levels, providing fine-grained control over performance versus compression ratio. One of the main advantages of Zstd is that the decompression speed is independent of the compression level. For data written once but read often, Zstd allows the use of the highest compression levels without a read performance penalty.

Even with frequent data updates, enabling compression often provides higher performance. One of the biggest advantages comes from the compressed ARC feature. ZFS's Adaptive Replacement Cache (ARC) caches the compressed version of the data in RAM, decompressing it each time. This allows the same amount of RAM to store more data and metadata, increasing the cache hit ratio.

ZFS offers 19 levels of Zstd compression, each offering incrementally more space savings in exchange for slower compression. The default level is `zstd-3` and offers greater compression than LZ4 without being much slower. Levels above 10 require large amounts of memory to compress each block and systems with less than 16 GB of RAM should not use them. ZFS uses a selection of the Zstd *fast* levels also, which get correspondingly faster but support lower compression ratios. ZFS supports `zstd-fast-1` through `zstd-fast-10`, `zstd-fast-20` through `zstd-fast-100` in increments of 10, and `zstd-fast-500` and `zstd-fast-1000` which provide minimal compression, but offer high performance.

If ZFS is not able to get the required memory to compress a block with Zstd, it will fall back to storing the block uncompressed. This is unlikely to happen except at the highest levels of Zstd on memory constrained systems. ZFS counts how often this has occurred since loading the ZFS module with `kstat.zfs.misc.zstd.compress_alloc_fail`.

23.4.15. Rewriting Existing Data

Note

`zfs rewrite` requires OpenZFS 2.4, first shipped in FreeBSD 15.0.

Properties like `compression`, `checksum`, `dedup`, and `copies` affect newly written data only. Changing them leaves existing blocks stored as they were, and applying the new values traditionally required a send/receive cycle or copying every file back and forth. `zfs rewrite` instead rewrites the blocks of existing files in place, as if they were atomically read and written back, so the current property values take effect without those workarounds.

The command operates on files and directories rather than dataset names. To apply a newly chosen compression algorithm to a whole dataset, recurse from its mount point:

```
# zfs set compression=zstd mypool/archive
# zfs rewrite -r /mypool/archive
```



Adding `-v` prints the name of every rewritten file. `-x` keeps the recursion from crossing mount points into child datasets, and `-o` and `-l` restrict the rewrite to a byte range within a file.



Property changes that would alter the logical block size, such as `recordsize`, have no effect on rewritten files.

Rewritten blocks are new blocks: snapshots taken before the rewrite keep referencing the old copies, so space usage can grow until those snapshots are destroyed. `zfs rewrite` does not work through snapshots for the same reason. Refer to [zfs-rewrite\(8\)](#) for details.

23.4.16. Deduplication

When enabled, [deduplication](#) uses the checksum of each block to detect duplicate blocks. When a new block is a duplicate of an existing block, ZFS writes a new reference to the existing data instead of the whole duplicate block. Tremendous space savings are possible if the data contains a lot of duplicated files or repeated information. Warning: deduplication requires a large amount of memory, and enabling compression instead provides most of the space savings without the extra cost.

`dedup` is a dataset property, not a pool property. To activate deduplication, set it on a dataset:

```
# zfs set dedup=on pool
```

This example sets the property on the root dataset of the pool *pool*, so every dataset in the pool inherits it. Limiting deduplication to the datasets that actually store duplicate-heavy data keeps the deduplication table smaller.

Deduplicating only affects new data written to the dataset. Merely activating this option will not deduplicate data already written. On FreeBSD 15.0, `zfs rewrite` can rewrite existing data so that it passes through deduplication. A pool with a freshly activated deduplication property will look like this example:

```
# zpool list
```

NAME	SIZE	ALLOC	FREE	CKPOINT	EXPANDSZ	FRAG	CAP	DEDUP	HEALTH	ALROOT
pool	2.84G	2.19M	2.83G	-	-	0%	0%	1.00x	ONLINE	-

The `DEDUP` column shows the actual rate of deduplication for the pool. A value of `1.00x` shows that data has not deduplicated yet. The next example copies some system binaries three times into different directories on the deduplicated pool created above.

```
# for d in dir1 dir2 dir3; do
> mkdir $d && cp -R /usr/bin $d &
> done
```

To observe deduplicating of redundant data, use:



```
# zpool list
```

NAME	SIZE	ALLOC	FREE	CKPOINT	EXPANDSZ	FRAG	CAP	DEDUP	HEALTH	ALTROOT
pool	2.84G	20.9M	2.82G	-	-	0%	0%	3.00x	ONLINE	-

The **DEDUP** column shows a factor of **3.00x**. Detecting and deduplicating copies of the data uses a third of the space. The potential for space savings can be enormous, but comes at the cost of having enough memory to keep track of the deduplicated blocks. ZFS stores an entry for every deduplicated block in the deduplication table (**DDT**). A general rule of thumb is 5-6 GB of RAM per 1 TB of deduplicated data. When the table no longer fits in memory, every write forces reads of table entries from disk, and performance degrades drastically.

Deduplication is not always beneficial when the data in a pool is not redundant. ZFS can show potential space savings by simulating deduplication on an existing pool:

```
# zdb -S pool
```

Simulated DDT histogram:

bucket	allocated				referenced			
refcnt	blocks	LSIZE	PSIZE	DSIZE	blocks	LSIZE	PSIZE	DSIZE
1	2.58M	289G	264G	264G	2.58M	289G	264G	264G
2	206K	12.6G	10.4G	10.4G	430K	26.4G	21.6G	21.6G
4	37.6K	692M	276M	276M	170K	3.04G	1.26G	1.26G
8	2.18K	45.2M	19.4M	19.4M	20.0K	425M	176M	176M
16	174	2.83M	1.20M	1.20M	3.33K	48.4M	20.4M	20.4M
32	40	2.17M	222K	222K	1.70K	97.2M	9.91M	9.91M
64	9	56K	10.5K	10.5K	865	4.96M	948K	948K
128	2	9.50K	2K	2K	419	2.11M	438K	438K
256	5	61.5K	12K	12K	1.90K	23.0M	4.47M	4.47M
1K	2	1K	1K	1K	2.98K	1.49M	1.49M	1.49M
Total	2.82M	303G	275G	275G	3.20M	319G	287G	287G

dedup = 1.05, compress = 1.11, copies = 1.00, dedup * compress / copies = 1.16

After **zdb -S** finishes analyzing the pool, it shows the space reduction ratio that activating deduplication would achieve. In this case, **1.16** is a poor space saving ratio mainly provided by compression. Activating deduplication on this pool would not save any amount of space, and is not worth the amount of memory required to enable deduplication. Using the formula $ratio = dedup * compress / copies$, system administrators can plan the storage allocation, deciding whether the workload will contain enough duplicate blocks to justify the memory requirements. If the data is reasonably compressible, the space savings may be good.

Fast dedup, a reworked implementation that lowers these costs, requires OpenZFS 2.3 or later, first available in FreeBSD 15.0. Fast dedup batches deduplication table updates in an on-disk log

and flushes them to the table in the background, reducing the random I/O that made classic deduplication slow. The `dedup_table_quota` pool property sets a limit on the on-disk size of the table; ZFS stops adding new entries once the pool reaches the limit, and with the default value of `auto` the size of a dedicated dedup vdev serves as the quota. The read-only `dedup_table_size` pool property reports the current size of the table. `zpool ddtprune -p 30 pool` removes the oldest 30% of the table entries that provide no savings because only a single block references them; `-d days` instead prunes single-reference entries older than the given number of days. `zpool status -DD pool` displays deduplication table statistics, including a histogram of blocks by reference count. Storing the table on a dedicated [dedup vdev](#) made of fast devices keeps lookups quick even when the table outgrows RAM.

Good practice is to enable compression first as compression also provides greatly increased performance. Enable deduplication in cases where savings are considerable and with enough available memory for the [DDT](#).

23.4.17. ZFS and Jails

Use `zfs jail` and the corresponding `jailed` property to delegate a ZFS dataset to a [Jail](#). `zfs jail jailid` attaches a dataset to the specified jail, and `zfs unjail` detaches it. To control the dataset from within a jail, set the `jailed` property. ZFS forbids mounting a jailed dataset on the host because it may have mount points that would compromise the security of the host.

A delegated dataset hands control of a whole subtree to the jail administrator, who can mount it, create child datasets, take snapshots, and change properties, all confined to that subtree. For this to work, the jail needs the `allow.mount` and `allow.mount.zfs` parameters enabled and the `enforce_statfs` parameter set to a value lower than 2. Refer to [jail\(8\)](#) for details on these parameters.

This example creates a dataset, marks it as jailed, and attaches it to the running jail *myjail*:

```
# zfs create mypool/jaildata
# zfs set jailed=on mypool/jaildata
# zfs jail myjail mypool/jaildata
```

The dataset then appears in `zfs list` inside the jail and is manageable there. `zfs unjail myjail mypool/jaildata` returns control to the host. See [zfs-jail\(8\)](#) for more information.

Starting with FreeBSD 15.0, [jail\(8\)](#) automates the attachment. List the datasets to delegate in the `zfs.dataset` jail parameter, and [jail\(8\)](#) attaches them when the jail starts. The datasets must already have `jailed=on` set, and the parameter requires `allow.mount.zfs`. An `/etc/jail.conf` entry using it looks like this:

```
myjail {
    path = "/usr/local/jails/myjail";
    host.hostname = "myjail.example.org";
```




```
exec.start = "/bin/sh /etc/rc";
exec.stop = "/bin/sh /etc/rc.shutdown";
allow.mount;
allow.mount.zfs;
enforce_statfs = 1;
zfs.dataset = "mypool/jaildata";
}
```

Setting the `zfs.mount_snapshot` jail parameter to 1 additionally lets users inside the jail access the contents of ZFS snapshots under the `.zfs` directory of the delegated file systems.

23.5. Delegated Administration

A comprehensive permission delegation system allows unprivileged users to perform ZFS administration functions. For example, if each user's home directory is a dataset, users need permission to create and destroy snapshots of their home directories. A user performing backups can get permission to use replication features. ZFS allows a usage statistics script to run with access to only the space usage data for all users. Delegating the ability to delegate permissions is also possible. Permission delegation is possible for each subcommand and most properties.

23.5.1. Delegating Dataset Creation

`zfs allow someuser create mydataset` gives the specified user permission to create child datasets under the selected parent dataset. A caveat: creating a new dataset involves mounting it. That requires setting the FreeBSD `vfs.usermount` [sysctl\(8\)](#) to `1` to allow non-root users to mount a file system. Another restriction aimed at preventing abuse: non-`root` users must own the mountpoint where the file system is to be mounted.

23.5.2. Delegating Permission Delegation

`zfs allow someuser allow mydataset` gives the specified user the ability to assign any permission they have on the target dataset, or its children, to other users. If a user has the `snapshot` permission and the `allow` permission, that user can then grant the `snapshot` permission to other users.

23.6. ZFS Native Encryption

ZFS supports native encryption of datasets and the data stored within them. This was not always the case, as previous solutions relied on FreeBSD's GELI-based encryption. While this approach is secure, it was not as easily portable to non-FreeBSD systems that lacked GELI support. With ZFS native encryption, encrypted datasets can be used on other systems that support this pool feature without relying on third-party components or operating system-native encryption.

Another benefit of ZFS native encryption is that datasets do not need to be decrypted for administrative tasks such as snapshots, replication, or scrub operations. ZFS data integrity checks



work just as well with natively encrypted datasets, and compression also works on encrypted datasets.

Different datasets on the same pool can use separate keys independently. Consider a central file server where different users store their data in encrypted home datasets. User A has a different key than User B. Both users can work side by side and choose to unencrypt their datasets as needed without knowing each other's decryption keys. The decryption method can also differ completely between users without affecting one another. This allows sensitive data from different users to be stored on the same pool while remaining protected by separate keys.

Before using ZFS native encryption, be aware of the following:

- Encryption is applied at the dataset level, not the pool level.
- Booting from encrypted ZFS datasets is not yet supported in FreeBSD's loader.
- Encryption can only be enabled when a dataset is created, not afterward.
- The block cipher, key length, and encryption mode cannot be changed after they have been set.
- ZFS does not encrypt metadata.

The last point may seem like a drawback compared to full-disk encryption. However, it provides greater flexibility by allowing different keys to protect datasets within the same pool instead of requiring a single key or passphrase to unlock the entire pool. Specifically, ZFS encrypts the following:

- file and zvol data,
- file attributes,
- ACLs,
- permission bits,
- directory listings,
- FUID mappings,
- userused/groupused data, and
- deduplicated data

Some ZFS metadata and information remains unencrypted, including the following:

- pool structure and name,
- dataset and snapshot names,
- dataset hierarchy,
- properties,



- file size,
- file holes, and
- deduplication tables

This unencrypted metadata allows ZFS to perform routine maintenance operations, such as `zpool scrub`, without first decrypting the data. As a result, ZFS can detect and repair corrupted encrypted data without accessing the unencrypted file contents.

To encrypt existing unencrypted data, copy it to a dataset with encryption enabled. This allows data to be migrated to a secure location within the pool, provided the passphrase is supplied to authorize access. Typically, this is done by loading the encryption key into memory after confirming knowledge of a secret such as a passphrase. Once the key has been loaded, the dataset contents become accessible for reading and writing. To re-secure the dataset, unload the key from memory. ZFS then makes the dataset inaccessible until the key is loaded again.

When encryption is enabled, some ZFS operations behave differently. ZFS applies compression before encryption to preserve compression ratios. While ZFS normally uses 256-bit checksums, encryption replaces them with a 128-bit checksum and a 128-bit Message Authentication Code (MAC) provided by the encryption suite. This provides additional protection against malicious data modification.

When deduplication is used together with encryption, ZFS performs deduplication only within the encrypted dataset, its snapshots, and its clones. This prevents information from leaking between encrypted datasets. The tradeoff is lower deduplication efficiency because ZFS cannot compare checksums across the entire pool. Even so, deduplication combined with encryption still reveals which blocks are identical and incurs additional CPU overhead for each block written.

Other limitations apply when using ZFS encryption. The `embedded_data` feature cannot be used with encryption. Datasets with encryption enabled also cannot have the `copies` property set to 3, because the implementation stores encrypted metadata in the location where the third copy would normally reside.

23.6.1. Creating an Encrypted Dataset

ZFS encryption is enabled by setting the `encryption=on` property when creating a dataset. Setting this property outside of a `zfs create` command does not enable encryption retroactively, as it only takes effect during dataset creation. In the following example, a passphrase is used when mounting the encrypted dataset after it has been created.

```
# zfs create -o encryption=on -o keyformat=passphrase -o keylocation=prompt zroot/secretdata
Enter passphrase:
Re-enter passphrase:
```

If the passphrase is shorter than eight characters, ZFS rejects it and does not create the dataset. After the passphrase has been entered correctly twice, ZFS creates and mounts the dataset in the

pool.

Any data stored in the encrypted dataset remains encrypted while it resides there. Copying data from an encrypted dataset to an unencrypted location decrypts the data during the copy operation. The newly created dataset can then be mounted at a different location if necessary.

zfs set mountpoint=/secretdata zroot/secretdata
echo "Hello FreeBSD!" > /secretdata/message
cat /secretdata/message
Hello FreeBSD!

As expected, the data is currently available for reading and writing by anyone who has access to the dataset.

23.6.2. Properties of Encrypted Datasets

Listing the ZFS native encryption properties for a dataset is done with the following command:

zfs get encryption,keylocation,keyformat zroot/secretdata
NAME PROPERTY VALUE SOURCE
zroot/secretdata encryption aes-256-gcm -
zroot/secretdata keylocation prompt local
zroot/secretdata keyformat passphrase -

The `encryption` property shows `aes-256-gcm`, which is currently the default encryption method used by ZFS. As indicated by the `SOURCE` column, neither this property nor the `keyformat` property can be changed using `zfs set` after the dataset has been created. See the [changing the encryption key](#) section for information on how to modify some of these properties. The following table lists the encryption-related properties used by ZFS:

encryptionroot	The dataset from which the current dataset inherits its encryption key. Dataset clones share the encryption key of their originating dataset.
keyformat	The format of the encryption key. Possible values are: <code>raw</code> , <code>hex</code> , and <code>passphrase</code> . Both <code>raw</code> and <code>hex</code> keys must be 32 bytes long and contain random values.
encryption	The cipher used for encryption, consisting of the block cipher, key length, and encryption mode. Both the <code>encryption</code> and <code>keyformat</code> properties must be specified when the dataset is created. Possible values for the <code>encryption</code>  property are:

	• <code>off</code> (default), • <code>on</code>, • <code>aes-128-ccm</code>, • <code>aes-192-ccm</code>, • <code>aes-256-ccm</code>, • <code>aes-128-gcm</code>, • <code>aes-192-gcm</code>, and • <code>aes-256-gcm</code> (current standard when <code>on</code> is set). This property cannot be changed once it has been set.
keystatus	Indicates whether an encryption key has been successfully loaded. Possible values are <code>none</code> , <code>available</code> , and <code>unavailable</code> .
keylocation	Specifies the location from which to load the encryption key. Possible values are: • <code>prompt</code>, • <code>file://path/to/file</code>, • <code>http://some.address</code>, and • <code>https://other.address</code>
pbkdf2iters	The number of iterations used when deriving an encryption key from a passphrase. This forces an attacker to perform a large number of computationally expensive hash operations. As computing power increases, this value should also be increased from its current default of 350000.

23.6.3. Unloading the Encryption Key

Protecting the data stored in an encrypted dataset requires two steps: first unmount the dataset, then unload the encryption key from memory. This distinction is important. An unmounted dataset is not protected if its encryption key remains loaded in memory. Always run `zfs unload-key` to protect the data. To access the dataset again, load the key and provide the passphrase as



described in the [loading the encryption key](#) section. ZFS prevents `zfs unload-key` from running while the dataset is still mounted.

```
# zfs unmount zroot/secretdata
# zfs unload-key zroot/secretdata
```

Running `zfs unload-key` multiple times results in an error indicating that the key has already been unloaded.

23.6.4. Loading the Encryption Key

Before an encrypted dataset can be used like any other dataset, its encryption key must be loaded and the passphrase provided. At this point, the `keystatus` property is still set to `unavailable`, and mounting the dataset fails because the key has not yet been loaded:

```
# zfs get keystatus zroot/secretdata
NAME          PROPERTY  VALUE      SOURCE
zroot/secretdata  keystatus  unavailable -
# zfs mount zroot/secretdata
cannot mount 'zroot/secretdata': encryption key not loaded
```

This indicates that the dataset is encrypted and requires an encryption key. To mount the dataset, run `zfs load-key` and provide the passphrase when prompted:

```
# zfs load-key zroot/secretdata
Enter passphrase for 'zroot/secretdata':
# zfs get keystatus zroot/secretdata
NAME          PROPERTY  VALUE      SOURCE
zroot/secretdata  keystatus  available  -
```

Next, mount the dataset to make it accessible through the filesystem hierarchy:

```
# zfs mount zroot/secretdata
# mount|grep secretdata
zroot/secretdata on /secretdata (zfs, local, noatime, nfsv4acls)
```

To load multiple encryption keys recursively, specify the `-r` option with `zfs load-key`. If a key has already been loaded, ZFS reports this with an error message.

```
# zfs load-key zroot/secretdata
```

Key load error: Key already loaded for 'zroot/secretdata'.

The `keystatus` property also confirms that the key has been loaded by reporting the value `available`. To verify whether a passphrase is correct without loading the key, use the `-n` option to perform a dry run.

23.6.5. Changing the Encryption Key

ZFS also supports changing encryption keys, such as replacing a passphrase. This operation does not require re-encrypting the dataset. Run the following command to change the encryption passphrase:

```
# zfs change-key zroot/secretdata
Enter new passphrase for 'zroot/secretdata':
Re-enter new passphrase for 'zroot/secretdata':
```

Running `zfs change-key` requires the current key to be loaded. If it is not, ZFS still prompts for the new key but then displays a warning that the current key has not been loaded.

The `zfs change-key` command can also modify the `keylocation`, `keyformat`, and `pbkdf2iters` properties by specifying them with the `-o` option. Running `zfs change-key` on an encrypted child dataset makes it an encryption root if it is not one already. To prevent this behavior and continue inheriting the parent's key, specify the `-i` option.

Be aware that changing the encryption key of a parent dataset also changes the key for child datasets that inherit it. Consequently, those child datasets continue to use the parent's encryption key. If a child dataset should use a different key, either specify a different `keyformat` when creating it or run `zfs change-key` on the child dataset. Doing so creates a new encryption root for the child and breaks its encryption inheritance from the parent.

ZFS clones of an encrypted dataset always use the encryption key of their origin dataset. As a result, the `keystatus`, `keyformat`, `keylocation`, and `pbkdf2iters` properties are not inherited like other dataset properties. Instead, they use the values defined by the encryption root. To determine the encryption root, use the read-only `encryptionroot` property.

If an attacker compromises an encryption key, changing the passphrase with `zfs change-key` does not necessarily protect existing or newly written data. New data continues to be encrypted with the same master key as the existing data. If an attacker obtains both a user key and its corresponding wrapped master key, running `zfs change-key` does not overwrite the previous master key on disk. As a result, the old master key may remain available for forensic analysis for an indeterminate period.

If a master key has been compromised, the preferred solution is to securely erase the underlying storage devices and create a new pool. Afterwards, restore the data from a backup to the new pool. Alternatively, create a new encrypted dataset, migrate the data using `zfs send` and `zfs` 

receive, and then run `zpool trim --secure` to erase the freed space. If the underlying hardware does not support secure TRIM, use `zpool initialize` instead.

23.6.6. Sending Encrypted Datasets

ZFS replication is based on sending dataset snapshots, which can also be created from encrypted datasets. The resulting snapshots remain encrypted.

```
# zfs snapshot zroot/secretdata@snap1
```

In addition to [sending encrypted backups over SSH](#), using the `-w` (raw) option with `zfs send` transfers the encrypted data blocks to the target pool. A raw send provides several advantages:

- the receiving system never sees plaintext data.
- the receiving system does not require the passphrase since no decryption takes place.
- Backups can be sent without first loading the encryption key.
- an untrusted system can receive the encrypted data but cannot decrypt or modify it without the encryption key.

To send the snapshot created above as a raw stream to the dataset `secret` in the pool `backup`, enter the following command:

```
# zfs send -w zroot/secretdata@snap1 | zfs recv backup/secret
```

Without the `-w` option, `zfs send` transmits the data in unencrypted form. The receiving system may then re-encrypt the data using a different key. However, doing so prevents future incremental raw sends to that destination.

23.6.7. Loading Encryption Keys at Boot

The `zfskeys rc(8)` script loads encryption keys at boot for encrypted datasets whose `keylocation` property points to a key file with a `file://` URL. The script runs after pool import and before mounting the file systems, so datasets handled this way mount automatically during the normal ZFS startup. The script does not handle datasets with `keylocation=prompt`; load their keys manually with `zfs load-key` after the system has booted.

To switch the dataset created earlier from a passphrase to a key file, generate 32 bytes of random data as the new key, restrict access to the key file, and change the encryption key of the dataset:

```
# dd if=/dev/random of=/root/secretdata.key bs=32 count=1
# chmod 600 /root/secretdata.key
```

```
# zfs change-key -o keyformat=raw -o keylocation=file:///root/secretdata.key zroot/secretdata
```

Enable the script in **/etc/rc.conf**:

```
# sysrc zfskeys_enable="YES"
```

By default, the script loads the keys of all encrypted datasets with a `file://` key location. To restrict key loading to specific datasets, list them in `zfskeys_datasets`, separated by spaces:

```
# sysrc zfskeys_datasets="zroot/secretdata"
```

⚠ Warning

Anyone able to read the key file can decrypt the dataset. Keep the key file owned by `root` with mode `600`, and store it on storage that is itself protected, such as an encrypted root file system or removable media that is only connected during boot.

23.7. Boot Environments

A boot environment is a bootable [clone](#) of the dataset tree that contains the operating system. On systems installed with the Root-on-ZFS layout, these datasets live under `zroot/R00T`, with the running system in `zroot/R00T/default`. Creating a boot environment snapshots and clones the root datasets, which completes in seconds and consumes almost no space until the environments start to diverge.

Manage boot environments with [bectl\(8\)](#). Create a new environment before a major change, such as an operating system or large package upgrade:

```
# bectl create beforeupgrade
# bectl list
```

BE	Active	Mountpoint	Space	Created
default	NR	/	2.43G	2026-07-11 11:26
beforeupgrade	-	-	328K	2026-07-11 11:31

In the `Active` column, `N` marks the environment in use now, and `R` marks the one that becomes active on reboot.

Perform the upgrade with [freebsd-update\(8\)](#) or [pkg\(8\)](#) as usual. The upgrade modifies the active environment, while `beforeupgrade` preserves the system as it was. If the upgraded system fails to boot or misbehaves, reboot and select `beforeupgrade` from the `Boot Environments` menu. 

of the FreeBSD loader. Selecting an environment in the loader menu affects the current boot only. To make the rollback permanent, activate the environment:

```
# bectl activate beforeupgrade
```

Running `bectl activate -t beName` instead activates an environment for the next boot only, which is useful when testing changes on a remote system.

To inspect or repair the contents of an inactive environment without booting it, mount it at a temporary location:

```
# bectl mount beforeupgrade  
/tmp/be_mount.c1Xk  
# bectl umount beforeupgrade
```

Destroy an environment that is no longer needed to reclaim the space it uses:

```
# bectl destroy beforeupgrade
```

💡 Tip

Creating a boot environment before every [freebsd-update\(8\)](#) upgrade or large [pkg\(8\)](#) operation provides an instant way back to a known-good system without restoring from backups.

23.7.1. Boot Environments and Applications

A boot environment contains only the datasets under `zroot/R00T`. On the default Root-on-ZFS layout that includes the whole root file system with `/usr/local` and `/var/db`, so installed packages and their metadata roll back together with the operating system. Datasets outside `zroot/R00T`, such as `zroot/home`, `zroot/var/log`, and `zroot/var/mail`, are shared: every environment sees the same files, and activating an older environment does not return them to an earlier state.

Plan the dataset layout around this split before relying on boot environments. Data that must survive a rollback, such as the databases of an application installed from packages, belongs on its own dataset outside `zroot/R00T`:

```
# zfs create -o mountpoint=/var/db/postgres zroot/pgdata
```

With this layout, rolling back to an older environment returns the database server binaries to their previous version while the database contents remain untouched. The reverse also holds: files written to shared datasets while testing a new environment persist after switching back, so boot environments do not replace [snapshots](#) of those datasets.

23.8. Advanced Topics

23.8.1. Tuning

Adjust tunables to make ZFS perform best for different workloads. FreeBSD exposes the ZFS kernel tunables as [sysctl\(8\)](#) variables under `vfs.zfs`; [zfs\(4\)](#) documents every tunable and its default value. Set values that must apply from boot in **/boot/loader.conf**; adjust runtime-changeable values with [sysctl\(8\)](#) and make them permanent in **/etc/sysctl.conf**.

- `vfs.zfs.arc.max` - Upper size of the [ARC](#). The default of `0` lets ZFS size the ARC automatically based on the amount of installed memory. Use a lower value if the system runs any other daemons or processes that may require memory; see [ARC Sizing and Monitoring](#) for guidance. Adjust this value at runtime with [sysctl\(8\)](#) and set it in **/boot/loader.conf** or **/etc/sysctl.conf**.
- `vfs.zfs.arc.min` - Lower size of the [ARC](#). The default of `0` lets ZFS choose a small minimum automatically. Increase this value to prevent other applications from pressuring out the entire ARC. Adjust this value at runtime with [sysctl\(8\)](#) and set it in **/boot/loader.conf** or **/etc/sysctl.conf**.
- `vfs.zfs.arc.meta_balance` - Balance between caching metadata and file data in the [ARC](#); values above `100` increasingly favor metadata, and the default is `500`. Increase this value if the workload involves operations on a large number of files and directories, or frequent metadata operations, at the cost of less file data fitting in the ARC. This tunable replaces `vfs.zfs.arc.meta_limit`, which OpenZFS 2.2 removed. Adjust this value at any time with [sysctl\(8\)](#).
- `vfs.zfs.vdev.min_auto_ashift` - Lower `ashift` (sector size) used automatically at pool creation time. The value is a power of two. The default value of `9` represents $2^9 = 512$, a sector size of 512 bytes. To avoid *write amplification* and get the best performance, set this value to the largest sector size used by a device in the pool.

See [Creating and Destroying Storage Pools](#) for why pools on 4 KB-sector drives should use an `ashift` of 12.

- `vfs.zfs.prefetch.disable` - Disable prefetch. A value of `0` enables and `1` disables it. The default is `0`. Prefetch works by reading larger blocks than requested into the [ARC](#) in hopes to soon need the data. If the workload has a large number of random reads, disabling prefetch may actually improve performance by reducing unnecessary reads. Adjust this value at any time with [sysctl\(8\)](#).
- `vfs.zfs.txg.timeout` - Upper number of seconds between [transaction groups](#). The current transaction group writes to the pool and a fresh transaction group starts if this



amount of time elapses since the previous transaction group. A transaction group may trigger earlier if writing enough data. The default value is 5 seconds. A larger value may improve read performance by delaying asynchronous writes, but this may cause uneven performance when writing the transaction group. Adjust this value at any time with `sysctl(8)`.

- `vfs.zfs.l2arc.write_max` - Limit the amount of data written to the [L2ARC](#) per second. This tunable extends the longevity of SSDs by limiting the amount of data written to the device. Adjust this value at any time with `sysctl(8)`.
- `vfs.zfs.l2arc.write_boost` - Adds the value of this tunable to `vfs.zfs.l2arc.write_max` and increases the write speed to the SSD until evicting the first block from the [L2ARC](#). This "Turbo Warmup Phase" reduces the performance loss from an empty L2ARC after a reboot. Adjust this value at any time with `sysctl(8)`.
- `vfs.zfs.l2arc.rebuild_enabled` - Make the contents of the [L2ARC](#) persist across reboots. With the default of `1`, ZFS rebuilds the L2ARC from the log blocks stored on the cache device when importing a pool, avoiding the long warm-up period an empty cache would otherwise need. Set this value to `0` to always start with an empty L2ARC.
- `vfs.zfs.vdev.max_active` - Upper number of I/O requests active on each device in the pool. A higher value keeps the device command queues full and may give higher throughput. A lower value reduces latency. Adjust this value at any time with `sysctl(8)`.
- `vfs.zfs.vdev.*_min_active` and `vfs.zfs.vdev.*_max_active` - Lower and upper number of concurrent I/O requests the scheduler issues to each device for every I/O class: `sync_read`, `sync_write`, `async_read`, `async_write`, `scrub`, and others. These per-class limits replace the delay-based [scrub](#) and [resilver](#) throttling found in older versions of ZFS. For example, increasing `vfs.zfs.vdev.scrub_max_active` lets a scrub use a larger share of each device's queue, finishing sooner at the cost of foreground I/O latency.
- `vfs.zfs.resilver_min_time_ms` - Lower amount of time, in milliseconds, that a [resilver](#) spends working between transaction group flushes; the default is `3000` (3 seconds). Increase this value to complete a resilver sooner when a degraded pool is at risk of losing another device, at the cost of slowing down normal pool operation. Adjust this value at any time with `sysctl(8)`.

23.8.2. ARC Sizing and Monitoring

ZFS keeps recently and frequently used data and metadata in RAM in the [ARC](#). The ARC grows on demand up to `vfs.zfs.arc.max` and shrinks again when other programs need memory, so on a dedicated storage server it is normal for most of the RAM to be in use. The ARC stores blocks in their compressed on-disk form and decompresses them on access. This *compressed ARC* lets the same amount of RAM cache more data, increasing the effective hit rate.

`top(1)` shows the current ARC size in its memory summary line. The `kstat.zfs.misc.arcstats` `sysctl(8)` tree exposes detailed statistics, including the current size, the target maximum, and hit and miss counters:





```
% sysctl kstat.zfs.misc.arcstats.size kstat.zfs.misc.arcstats.c_max kstat.zfs.misc.arcstats.hits kstat.zfs.misc.arcstats.size: 8963041280
kstat.zfs.misc.arcstats.c_max: 15843122176
kstat.zfs.misc.arcstats.hits: 954912441
kstat.zfs.misc.arcstats.misses: 21356847
```

[filesystems/zfs-stats](#) summarizes these statistics in a human-readable report.

The ARC yields memory under pressure, but on systems that share RAM between ZFS and other large consumers such as databases, virtual machines, or jails, the competition can cause uneven performance on both sides. Capping the ARC leaves a predictable amount of memory for the other workloads. For example, to limit the ARC to 8 GB, add this line to **/boot/loader.conf**:

```
vfs.zfs.arc.max="8G"
```

Changing `vfs.zfs.arc.max` with [sysctl\(8\)](#) takes effect immediately, although shrinking a warm ARC frees memory gradually.

23.8.3. Synchronous Writes, the ZIL, and SLOG

Every pool has a [ZIL](#). Applications that request synchronous semantics with [fsync\(2\)](#) or the `O_SYNC` flag do not get an acknowledgment until ZFS commits the corresponding intent log record to stable storage. By default the ZIL lives on the pool's regular vdevs, so every synchronous write competes with normal pool I/O. Asynchronous writes bypass the ZIL and are written with the next [transaction group](#).

A dedicated [log](#) vdev, often called a *SLOG* (separate log), moves the ZIL to a fast device:

```
# zpool add mypool log mirror nda0 nda1
```



A SLOG helps only workloads that generate a lot of synchronous writes, such as NFS servers and databases. It makes no difference for purely asynchronous workloads.

A few gigabytes of SLOG capacity is plenty: the ZIL holds at most a few seconds of incoming writes, roughly one transaction group's worth, before ZFS writes the data to its final location in the pool. Choose SSDs with power-loss protection and low sustained write latency, and mirror the log devices. ZFS reads the ZIL only after a crash, so losing an unmirrored SLOG together with a system crash loses the writes that were acknowledged but not yet committed to the pool.

The `sync` dataset property controls synchronous write behavior:

- `sync=standard` (the default) commits synchronous writes to the ZIL and asynchronous writes with the next transaction group.



- `sync=always` treats every write as synchronous, trading performance for safety that helps only applications which fail to request synchronous writes when they should.
- `sync=disabled` treats every write as asynchronous.

Warning

With `sync=disabled`, ZFS acknowledges synchronous writes before they reach stable storage. The pool itself stays consistent, but after a crash or power failure, applications and NFS clients silently lose up to a few seconds of data they believed was safely written. Do not use `sync=disabled` for databases, NFS exports, or any other data that matters.

The `logbias` property tunes how ZFS uses the log: `logbias=latency` (the default) favors the SLOG to minimize commit latency, while `logbias=throughput` writes synchronous data directly to the main pool, reserving SLOG capacity for latency-sensitive datasets. See [zfsprops\(7\)](#) for details.

23.8.4. Direct I/O

Note

Direct I/O requires OpenZFS 2.3 or later, first available in FreeBSD 15.0.

The `direct` dataset property controls how ZFS handles requests made with the `O_DIRECT` flag:

- `direct=standard` (the default) honors `O_DIRECT`: properly aligned reads and writes bypass the [ARC](#).
- `direct=always` treats every properly aligned read or write as a direct request.
- `direct=disabled` silently ignores `O_DIRECT` and handles every request through the ARC.

Direct I/O avoids the memory copies and caching overhead of the ARC. On pools of very fast NVMe devices this overhead can dominate the cost of the I/O itself, and applications that maintain their own caches, such as databases, waste RAM by caching the same data twice. For such workloads, direct I/O can increase throughput and reduce memory use.

ZFS still verifies checksums for direct reads and writes, so data integrity protection is unchanged. Direct writes must be aligned to the dataset `recordsize`; ZFS silently redirects the unaligned portion of a request through the ARC. See [zfsprops\(7\)](#) for the full alignment rules and restrictions.

23.8.5. `recordsize` and `volblocksize`



The `recordsize` property sets the largest block ZFS uses to store a file in a dataset, 128 KB by default. Matching the block size to the workload's I/O pattern avoids reading and writing more data than the application asked for.

For databases that perform small random reads and writes, set `recordsize` to the database page size, typically 8 KB or 16 KB, before creating the database files. For datasets storing large media files or backup streams that are read and written sequentially, `recordsize=1M` reduces metadata overhead and improves compression. Volumes use the `volblocksize` property instead, which defaults to 16 KB and cannot change after creating the volume.

Changing `recordsize` affects newly written files only; existing files keep their original block size. Not even `zfs rewrite` applies a new `recordsize` to existing files, as property changes that affect the logical block size have no effect on rewritten blocks; copy the files anew to store them with the new value.

23.9. Further Resources

- [OpenZFS Documentation](#), including the Performance and Tuning guides
- [OpenZFS](#)
- [zfs\(4\)](#) describes every kernel tunable, [zpoolconcepts\(7\)](#) describes vdev types and pool layout, and [zfsprops\(7\)](#) describes every dataset property
- Snapshot and replication automation tools from the Ports Collection:
 - [filesystems/zrepl](#) - a replication daemon supporting push and pull setups, automatic snapshot management, and resumable transfers
 - [sysutils/sanoid](#) - policy-driven snapshot management, with the bundled `syncoid` handling replication over SSH
 - [filesystems/zfstools](#) - automatic snapshot rotation in the style of the OpenSolaris auto-snapshot service, run from [cron\(8\)](#)
 - [filesystems/zap](#) - simple snapshot maintenance and remote replication, configured through ZFS properties instead of a configuration file

23.10. ZFS Features and Terminology

ZFS is fundamentally different from traditional file systems. ZFS combines the roles of file system and volume manager, enabling the addition of new storage devices to a live system and making the new space available on the existing file systems in that pool at once. By combining the traditionally separate roles, ZFS overcomes previous limitations that prevented RAID groups from growing. A *vdev* is a top level device in a pool and can be a simple disk or a RAID transformation such as a mirror or RAID-Z array. ZFS file systems (called *datasets*) each have access to the combined free space of the entire pool. Used blocks from the pool decrease the space available to each file system. This approach avoids the common pitfall with extensive partitioning where free space becomes fragmented across the partitions.



pool	A storage <i>pool</i> is the most basic building block of ZFS. A pool consists of one or more vdevs, the underlying devices that store the data. A pool is then used to create one or more file systems (datasets) or block devices (volumes). These datasets and volumes share the pool of remaining free space. Each pool is uniquely identified by a name and a GUID. Pools created by modern OpenZFS are at pool version 5000; from that version on, individual capabilities are managed by <i>feature flags</i> rather than the version number. See zpool-features(7) and Upgrading a Storage Pool for details.
vdev Types	A pool consists of one or more vdevs, which themselves are a single disk or a group of disks, transformed to a RAID. When using a lot of vdevs, ZFS spreads data across the vdevs to increase performance and maximize usable space. All vdevs must be at least 64 MB in size. • <i>Disk</i> - The most basic vdev type is a standard block device. This can be an entire disk (such as <code>/dev/ada0</code>) or a partition (<code>/dev/ada0p3</code>). On FreeBSD, there is no performance penalty for using a partition rather than the entire disk. This differs from recommendations made by the Solaris documentation.  Caution Using an entire disk as part of a bootable pool is strongly discouraged, as this may render the pool unbootable. Likewise, you should not use an entire disk as part of a mirror or RAID-Z vdev. Reliably determining the size of an unpartitioned disk at boot time is impossible and there's no place to put in boot code. • <i>File</i> - Regular files may make up ZFS pools, which is useful for testing and experimentation. Use the full path to the file as the device path in <code>zpool create</code>. • <i>Mirror</i> - When creating a mirror, specify the <code>mirror</code> keyword followed by the list of member devices for the mirror. A mirror consists of two or more devices, writing all data to all member devices. A mirror vdev will hold as much data as its smallest member. A mirror vdev can withstand the failure of all but one of its members without losing any data.



Note

To upgrade a regular single disk vdev to a mirror vdev at any time, use `zpool attach`.

- *RAID-Z* - ZFS uses RAID-Z, a variation on standard RAID-5 that offers better distribution of parity and eliminates the "RAID-5 write hole" in which the data and parity information become inconsistent after an unexpected restart. ZFS supports three levels of RAID-Z which provide varying levels of redundancy in exchange for decreasing levels of usable storage. ZFS uses RAID-Z1 through RAID-Z3 based on the number of parity devices in the array and the number of disks which can fail before the pool stops being operational.

In a RAID-Z1 configuration with four disks, each 1 TB, usable storage is 3 TB and the pool will still be able to operate in degraded mode with one faulted disk. If another disk goes offline before replacing and resilvering the faulted disk would result in losing all pool data.

In a RAID-Z3 configuration with eight disks of 1 TB, the volume will provide 5 TB of usable space and still be able to operate with three faulted disks. The traditional recommendation is no more than nine disks in a single vdev. If more disks make up the configuration, the recommendation is to divide them into separate vdevs and stripe the pool data across them.

A configuration of two RAID-Z2 vdevs consisting of 8 disks each would create something like a RAID-60 array. A RAID-Z group's storage capacity is about the size of the smallest disk multiplied by the number of non-parity disks. Four 1 TB disks in RAID-Z1 has an effective size of about 3 TB, and an array of eight 1 TB disks in RAID-Z3 will yield 5 TB of usable space.

Growing a raidz vdev one disk at a time is possible with [RAID-Z expansion](#), which requires OpenZFS 2.3 or later, first available in FreeBSD 15.0.

- *dRAID* - A variant of RAID-Z that distributes data, parity, and spare capacity across all member disks using fixed-width stripes. Instead of dedicated hot spare disks, dRAID provides *distributed spares*: spare capacity spread over every member disk, allowing a failed disk to rebuild sequentially onto all remaining disks at once and restoring redundancy much faster than a traditional resilver onto a single spare. Specify a dRAID 

vdev as `draid[parity][:datad][:children][:sparess]`, as described in [zpoolconcepts\(7\)](#). See [dRAID Pools](#) for details.

- *Spare* - ZFS has a special pseudo-vdev type for keeping track of available hot spares. When [zfsd\(8\)](#) is enabled, ZFS deploys installed hot spares automatically to replace failed devices; otherwise, configure them manually to replace the failed device using `zpool replace`. See [Hot Spares and Automatic Replacement with zfsd](#) for details.
- *Log* - Every pool has a ZFS Intent Log ([ZIL](#)), stored on the regular pool devices by default. A dedicated log vdev, often called a SLOG, moves the intent log to a separate device, typically an SSD. Having a dedicated log device improves the performance of applications with a high volume of synchronous writes like databases. Mirroring of log devices is possible, but RAID-Z is not supported. If using a lot of log devices, writes will be load-balanced across them. See [Synchronous Writes, the ZIL, and SLOG](#) for details.
- *Cache* - Adding a cache vdev to a pool will add the storage of the cache to the [L2ARC](#). Mirroring cache devices is impossible. Since a cache device stores only new copies of existing data, there is no risk of data loss.
- *Special/Dedup* - Allocation class vdevs that store pool metadata, optionally small file blocks (controlled by the `special_small_blocks` dataset property), and deduplication tables on fast devices such as SSDs, speeding up a pool of slower disks. These vdevs are pool-critical: losing a special or dedup vdev loses the pool, so give them the same level of redundancy as the data vdevs. See [Special Allocation Classes](#) for details.

Transaction Group (TXG)

Transaction Groups are the way ZFS groups blocks changes together and writes them to the pool. Transaction groups are the atomic unit that ZFS uses to ensure consistency. ZFS assigns each transaction group a unique 64-bit consecutive identifier. There can be up to three active transaction groups at a time, one in each of these three states:

- * *Open* - A new transaction group begins in the open state and accepts new writes. There is always a transaction group in the open state, but the transaction group may refuse new writes if it has reached a limit. Once the open transaction group has reached a limit, or reaching the `vfs.zfs.txg.timeout`, the transaction group advances to the next state.
- * *Quiescing* - A short state that allows any pending operations to finish without blocking the creation of a new open transaction group. Once all the transactions in the group have completed, the transaction group advances to the final state.
- * *Syncing* - Write all the data in the transaction group to stable



	storage. This process will in turn change other data, such as metadata and space maps, that ZFS will also write to stable storage. The process of syncing involves several passes. On the first and biggest, all the changed data blocks; next come the metadata, which may take several passes to complete. Since allocating space for the data blocks generates new metadata, the syncing state cannot finish until a pass completes that does not use any new space. The syncing state is also where <i>synctasks</i> complete. Synctasks are administrative operations such as creating or destroying snapshots and datasets that complete the uberblock change. Once the sync state completes the transaction group in the quiescing state advances to the syncing state. All administrative functions, such as <code>snapshot</code> write as part of the transaction group. ZFS adds a created synctask to the open transaction group, and that group advances as fast as possible to the syncing state to reduce the latency of administrative commands.
Adaptive Replacement Cache (ARC)	ZFS uses an Adaptive Replacement Cache (ARC), rather than a more traditional Least Recently Used (LRU) cache. An LRU cache is a simple list of items in the cache, sorted by how recently object was used, adding new items to the head of the list. When the cache is full, evicting items from the tail of the list makes room for more active objects. An ARC consists of four lists; the Most Recently Used (MRU) and Most Frequently Used (MFU) objects, plus a ghost list for each. These ghost lists track evicted objects to prevent adding them back to the cache. This increases the cache hit ratio by avoiding objects that have a history of occasional use. Another advantage of using both an MRU and MFU is that scanning an entire file system would evict all data from an MRU or LRU cache in favor of this freshly accessed content. With ZFS, there is also an MFU that tracks the most frequently used objects, and the cache of the most commonly accessed blocks remains.
L2ARC	L2ARC is the second level of the ZFS caching system. RAM stores the primary ARC. Since the amount of available RAM is often limited, ZFS can also use cache vdevs. Solid State Disks (SSDs) are often used as these cache devices due to their higher speed and lower latency compared to traditional spinning disks. L2ARC is entirely optional, but having one will increase read speeds for cached files on the SSD instead of having to read from the regular disks. L2ARC can also speed up deduplication because a deduplication table (DDT) that does not fit in RAM but does fit in the L2ARC will be much faster than a DDT that must read from disk. Since OpenZFS 2.0, the L2ARC is persistent: after a reboot, ZFS rebuilds the cache contents from the headers stored on the cache device instead of starting with an empty cache. Limits on the data rate added to the cache devices prevent prematurely wearing out SSDs with extra



	writes. Until the cache is full (the first block evicted to make room), writes to the L2ARC limit to the sum of the write limit and the boost limit, and afterwards limit to the write limit. A pair of <code>sysctl(8)</code> values control these rate limits: <code>vfs.zfs.l2arc.write_max</code> controls the number of bytes written to the cache per second, while <code>vfs.zfs.l2arc.write_boost</code> adds to this limit until the first eviction from the cache. See Tuning for details.
ZIL	The ZFS Intent Log (ZIL) is the on-disk record that ZFS uses to replay synchronous writes after a crash. Every pool has a ZIL; by default it occupies a small amount of space on the regular pool devices. When an application requests a synchronous write (a guarantee that the data is stored to disk rather than merely cached for later writes), ZFS commits the data to the ZIL first and flushes it out to the regular storage with the next transaction group. Moving the ZIL to a dedicated log vdev (SLOG) on a device faster than the main storage, such as an SSD, greatly reduces latency and improves performance. Synchronous workloads like databases will profit from a fast ZIL alone. Regular asynchronous writes such as copying files will not use the ZIL at all. See Synchronous Writes, the ZIL, and SLOG for details.
Copy-On-Write	Unlike a traditional file system, ZFS writes a different block rather than overwriting the old data in place. When completing this write the metadata updates to point to the new location. When a shorn write (a system crash or power loss in the middle of writing a file) occurs, the entire original contents of the file are still available and ZFS discards the incomplete write. This also means that ZFS does not require a <code>fsck(8)</code> after an unexpected shutdown.
Dataset	<i>Dataset</i> is the generic term for a ZFS file system, volume, snapshot or clone. Each dataset has a unique name in the format <i>poolname/path@snapshot</i>. The root of the pool is a dataset as well. Child datasets have hierarchical names like directories. For example, <i>mypool/home</i>, the home dataset, is a child of <i>mypool</i> and inherits properties from it. Expand this further by creating <i>mypool/home/user</i>. This grandchild dataset will inherit properties from the parent and grandparent. Set properties on a child to override the defaults inherited from the parent and grandparent. Administration of datasets and their children can be delegated.
File system	A ZFS dataset is most often used as a file system. Like most other file systems, a ZFS file system mounts somewhere in the systems directory hierarchy and



	contains files and directories of its own with permissions, flags, and other metadata.
Volume	ZFS can also create volumes, which appear as disk devices. Volumes have a lot of the same features as datasets, including copy-on-write, snapshots, clones, and checksumming. Volumes can be useful for running other file system formats on top of ZFS, such as UFS virtualization, or exporting iSCSI extents.
Snapshot	The copy-on-write (COW) design of ZFS allows for nearly instantaneous, consistent snapshots with arbitrary names. After taking a snapshot of a dataset, or a recursive snapshot of a parent dataset that will include all child datasets, new data goes to new blocks, but without reclaiming the old blocks as free space. The snapshot contains the original file system version and the live file system contains any changes made since taking the snapshot using no other space. New data written to the live file system uses new blocks to store this data. The snapshot will grow as the blocks are no longer used in the live file system, but in the snapshot alone. Mounting these snapshots read-only allows recovering previous file versions. A rollback of a live file system to a specific snapshot is possible, undoing any changes that took place after taking the snapshot. Each block in the pool has a reference counter which keeps track of the snapshots, clones, datasets, or volumes use that block. As files and snapshots get deleted, the reference count decreases, reclaiming the free space when no longer referencing a block. Marking snapshots with a hold means any attempt to destroy it returns an <code>EBUSY</code> error. Each snapshot can have holds with a unique name each. The release command removes the hold so the snapshot can be deleted. Snapshots, cloning, and rolling back works on volumes, but independently mounting does not.
Clone	Cloning a snapshot is also possible. A clone is a writable version of a snapshot, allowing the file system to fork as a new dataset. As with a snapshot, a clone initially consumes no new space. As new data written to a clone uses new blocks, the size of the clone grows. When blocks are overwritten in the cloned file system or volume, the reference count on the previous block decreases. Removing the snapshot upon which a clone bases is impossible because the clone depends on it. The snapshot is the parent, and the clone is the child. Clones can be <i>promoted</i>, reversing this dependency and making the clone the parent and the previous parent the child. This operation requires no new space. Since the amount of space used by the parent and child reverses, it may affect existing quotas and reservations.



Bookmark	A bookmark records the point in time at which a snapshot was taken, without keeping any of the snapshot data. Bookmarks serve as the source of incremental <code>zfs send</code> streams, making it possible to destroy the snapshot on the sending side while retaining the ability to send further increments. Create a bookmark from an existing snapshot with <code>zfs bookmark</code>. See Bookmarks for details.
Hold	A hold is a named tag placed on a snapshot that prevents its destruction. Attempts to destroy a held snapshot fail with an <code>EBUSY</code> error until releasing every hold with <code>zfs release</code>. Holds protect snapshots that backup and replication tools still depend on. See Snapshot Holds for details.
Checksum	Every block is also checksummed. The checksum algorithm used is a per-dataset property, see <code>set</code>. The checksum of each block is transparently validated when read, allowing ZFS to detect silent corruption. If the data read does not match the expected checksum, ZFS will attempt to recover the data from any available redundancy, like mirrors or RAID-Z. Trigger a validation of all checksums with <code>scrub</code>. Checksum algorithms include: * <code>fletcher2</code> * <code>fletcher4</code> * <code>sha256</code> * <code>sha512</code> * <code>skein</code> * <code>edonr</code> * <code>blake3</code> (since FreeBSD 14.0) The <code>fletcher</code> algorithms are faster, but the cryptographic hashes (<code>sha256</code>, <code>sha512</code>, <code>skein</code>, <code>edonr</code>, and <code>blake3</code>) have a much lower chance of collisions at the cost of some performance. The default value of <code>on</code> currently selects <code>fletcher4</code>; deduplication requires a cryptographic hash such as <code>sha256</code>. Deactivating checksums is possible, but strongly discouraged.
Compression	Each dataset has a compression property. Since OpenZFS 2.2, new datasets have compression enabled by default (<code>compression=on</code>, which selects LZ4). This causes compression of all new data written to the dataset. Beyond a reduction in space used, read and write throughput often increases because fewer blocks need reading or writing. * <i>LZ4</i> - Added in ZFS pool version 5000 (feature flags), LZ4 is the algorithm that <code>compression=on</code> selects and the recommended general-purpose choice. LZ4 works about 50% faster than LZJB when operating on compressible data, and is over three times faster when operating on uncompressible data. LZ4 also decompresses about 80% faster than LZJB. On modern CPUs, LZ4 can often compress at over 500 MB/s, and decompress at over 1.5 GB/s (per single CPU core).



	* <i>ZSTD</i> - Zstandard offers higher compression ratios than LZ4 with a configurable trade-off between speed and compression, ranging from the fast <code>zstd-fast</code> variants to <code>zstd-19</code> for maximum space savings. See Zstandard Compression for details. * <i>LZJB</i> - A legacy compression algorithm, created by Jeff Bonwick (one of the original creators of ZFS). LZJB offers good compression with less CPU overhead compared to GZIP. LZ4 has replaced it as the default algorithm; prefer LZ4 or ZSTD for new datasets. * <i>GZIP</i> - A popular stream compression algorithm available in ZFS. One of the main advantages of using GZIP is its configurable level of compression. When setting the <code>compress</code> property, the administrator can choose the level of compression, ranging from <code>gzip1</code>, the lowest level of compression, to <code>gzip9</code>, the highest level of compression. This gives the administrator control over how much CPU time to trade for saved disk space. * <i>ZLE</i> - Zero Length Encoding is a special compression algorithm that compresses continuous runs of zeros alone. This compression algorithm is useful when the dataset contains large blocks of zeros.
Copies	When set to a value greater than 1, the <code>copies</code> property instructs ZFS to maintain copies of each block in the <code>crossref:zfs[zfs-term-filesystem,file system]</code> or volume . Setting this property on important datasets provides added redundancy from which to recover a block that does not match its checksum. In pools without redundancy, the copies feature is the single form of redundancy. The copies feature can recover from a single bad sector or other forms of minor corruption, but it does not protect the pool from the loss of an entire disk.
Deduplication	Checksums make it possible to detect duplicate blocks when writing data. With deduplication, the reference count of an existing, identical block increases, saving storage space. ZFS keeps a deduplication table (DDT) in memory to detect duplicate blocks. The table contains a list of unique checksums, the location of those blocks, and a reference count. When writing new data, ZFS calculates checksums and compares them to the list. When finding a match it uses the existing block. Using the SHA256 checksum algorithm with deduplication provides a secure cryptographic hash. Deduplication is tunable. If <code>dedup</code> is <code>on</code> , then a matching checksum means that the data is identical. Setting <code>dedup</code> to <code>verify</code> , ZFS performs a byte-for-byte check on the data ensuring they are actually identical. If the data is not identical, ZFS will note the hash collision and store the two blocks separately. As the DDT must store the hash of each unique block, it consumes a large amount of memory. A general



	rule of thumb is 5-6 GB of RAM per 1 TB of deduplicated data. In situations not practical to have enough RAM to keep the entire DDT in memory, performance will suffer greatly as the DDT must read from disk before writing each new block. Deduplication can use L2ARC to store the DDT, providing a middle ground between fast system memory and slower disks. Consider using compression instead, which often provides nearly as much space savings without the increased memory. <i>Fast dedup</i> batches DDT updates in a log, allows pruning stale entries with <code>zpool ddtprune</code>, and caps the table size with the <code>dedup_table_quota</code> pool property; it requires OpenZFS 2.3 or later, first available in FreeBSD 15.0. See Deduplication for details.
Block Cloning	Block cloning allows a file copy to reference the existing on-disk blocks of the source file instead of duplicating them, so the copy completes almost instantly and consumes no extra space until either file changes. On FreeBSD, <code>cp(1)</code> uses <code>copy_file_range(2)</code>, so ordinary file copies become block clones when the feature is enabled (the default on FreeBSD 15.0). The <code>bcloned</code>, <code>bclonesaved</code>, and <code>bcloneratio</code> pool properties report the resulting space savings. See Block Cloning for details.
Scrub	Instead of a consistency check like <code>fsck(8)</code>, ZFS has <code>scrub</code>. <code>scrub</code> reads all data blocks stored on the pool and verifies their checksums against the known good checksums stored in the metadata. A periodic check of all the data stored on the pool ensures the recovery of any corrupted blocks before needing them. A scrub is not required after an unclean shutdown, but good practice is at least once every month. ZFS verifies the checksum of each block during normal use, but a scrub makes certain to check even infrequently used blocks for silent corruption. ZFS improves data security in archival storage situations. Pause a running scrub with <code>zpool scrub -p</code> and resume it later by running <code>zpool scrub</code> again. See Scrubbing a Pool for details.
Checkpoint	A checkpoint preserves the entire state of a pool at a single point in time, including pool-wide configuration changes that snapshots cannot capture. Rewinding to a checkpoint with <code>zpool import --rewind-to-checkpoint</code> undoes everything that happened after taking it, such as enabling pool features, adding or removing vdevs, or destroying datasets. A pool has at most one checkpoint at a time. See Pool Checkpoints for details.
TRIM	TRIM notifies SSDs and other flash-based storage which blocks the pool no longer uses, allowing the device to erase them in advance and sustain write



	performance. Run <code>zpool trim</code> manually, or set the <code>autotrim</code> pool property to <code>on</code> to trim freed space continuously. See TRIM and Initialization for details.
Dataset Quota	ZFS provides fast and accurate dataset, user, and group space accounting as well as quotas and space reservations. This gives the administrator fine grained control over space allocation and allows reserving space for critical file systems. ZFS supports different types of quotas: the dataset quota, the reference quota (refquota), the user quota, and the group quota. Quotas limit the total size of a dataset and its descendants, including snapshots of the dataset, child datasets, and the snapshots of those datasets. Note Volumes do not support quotas, as the <code>volsize</code> property acts as an implicit quota.
Reference Quota	A reference quota limits the amount of space a dataset can consume by enforcing a hard limit. This hard limit includes space referenced by the dataset alone and does not include space used by descendants, such as file systems or snapshots.
User Quota	User quotas are useful to limit the amount of space used by the specified user.
Group Quota	The group quota limits the amount of space that a specified group can consume.
Dataset Reservation	The <code>reservation</code> property makes it possible to guarantee an amount of space for a specific dataset and its descendants. This means that setting a 10 GB reservation on storage/home/bob prevents other datasets from using up all free space, reserving at least 10 GB of space for this dataset. Unlike a regular <code>refreservation</code>, space used by snapshots and descendant datasets does count against the reservation. Reservations of any sort are useful in situations such as planning and testing  the suitability of disk space allocation in a new system, or ensuring that enough

	space is available on file systems for audio logs or system recovery procedures and files.
Reference Reservation	The <code>refreservation</code> property makes it possible to guarantee an amount of space for the use of a specific dataset <i>excluding</i> its descendants. This means that setting a 10 GB reservation on storage/home/bob, and another dataset tries to use the free space, reserving at least 10 GB of space for this dataset. In contrast to a regular <code>reservation</code>, space used by snapshots and descendant datasets is not counted against the reservation. For example, if taking a snapshot of storage/home/bob, enough disk space other than the <code>refreservation</code> amount must exist for the operation to succeed. Descendants of the main data set are not counted in the <code>refreservation</code> amount and so do not encroach on the space set.
Boot Environment	A boot environment is a bootable clone of the datasets that hold the FreeBSD system, managed with <code>bectl(8)</code>. Creating a boot environment before a system upgrade makes it possible to boot the previous, known-good system from the loader menu if the upgrade causes problems. See Boot Environments for details.
Resilver	When replacing a failed disk, ZFS must fill the new disk with the lost data. <i>Resilvering</i> is the process of writing that data to the replacement device: mirrors resilver by copying the data from the surviving members, while RAID-Z reconstructs it from the parity information distributed across the remaining drives. Unlike a traditional RAID rebuild, resilvering copies allocated data alone rather than every sector of the device. A <i>sequential resilver</i> (<code>zpool attach -s</code> or <code>zpool replace -s</code>) rebuilds mirror and dRAID vdevs even faster by copying blocks in disk order, skipping checksum verification and deferring it to an automatically scheduled scrub. See Dealing with Failed Devices for details.
Online	A pool or vdev in the <code>Online</code> state has its member devices connected and fully operational. Individual devices in the <code>Online</code> state are functioning.
Offline	The administrator puts individual devices in an <code>Offline</code> state if enough redundancy exists to avoid putting the pool or vdev into a <code>Faulted</code> state. An administrator may choose to offline a disk in preparation for replacing it, or to make it easier to identify.



Degraded	A pool or vdev in the Degraded state has one or more disks that disappeared or failed. The pool is still usable, but if other devices fail, the pool may become unrecoverable. Reconnecting the missing devices or replacing the failed disks will return the pool to an Online state after the reconnected or new device has completed the Resilver process.
Faulted	A pool or vdev in the Faulted state is no longer operational. Accessing the data is no longer possible. A pool or vdev enters the Faulted state when the number of missing or failed devices exceeds the level of redundancy in the vdev. If reconnecting missing devices the pool will return to an Online state. Insufficient redundancy to compensate for the number of failed disks loses the pool contents and requires restoring from backups.
Unavail	A device in the Unavail state cannot be opened, because it is missing or inaccessible. The pool continues operating if enough redundancy exists to compensate, running in a Degraded state. If a top-level vdev is Unavail , accessing the pool contents is impossible.
Removed	A device in the Removed state was physically detached while the system was running. Device removal detection is hardware-dependent and unsupported on some platforms. When the device reconnects, zfsd(8) brings it back online; if the device does not return, zfsd activates an available hot spare in its place.

Last modified on: July 18, 2026 by [Sergio Carlavilla Delgado](#)



[Get FreeBSD](#)

[Code of Conduct](#)

[Security Advisories](#)

Documentation

[Documentation portal](#)

[Manual pages](#)

[Presentations and papers](#)

[Previous versions](#)

[4.4BSD Documents](#)

[Wiki](#)

Community

[Get involved](#)

[Community forum](#)

[Mailing lists](#)

[IRC Channels](#)

[Bug Tracker](#)

Legal

[Donations](#)

[Licensing](#)

[Privacy Policy](#)

[Legal notices](#)

© 1994-2026 The FreeBSD Project. All rights reserved

Made with  by the FreeBSD Community

